

INTERNATIONAL BACCALAUREATE

PHYSICS HL

EXTENDED ESSAY

Candidate code: lxw775

How does the frequency of an input sine wave f^* , with background white noise, affect the signal-to-noise ratio (SNR) after noise reduction through a RC low-pass filter?

Word count: 4000

1 Introduction

1.1 RC Circuits and Low-Pass Filters

A *resistor-capacitor circuit*, abbreviated as a *RC circuit*, is a circuit which contains a resistor and a capacitor connected in series.

A *low-pass filter* is a filter that passes signals with a frequency lower than a selected *cutoff frequency*, f_c , and attenuates signals with frequencies higher than f_c (see [1]). An ideal low-pass filter would allow all signals with frequency lower than f_c to completely pass through, and block all signals with frequency higher than f_c .

The *transfer function* of a system measures the magnitude response as a function of input frequency (see [2]). The magnitude response is defined as the ratio between the output magnitude and the input magnitude. For example, an ideal low-pass filter has magnitude response 1 for all frequencies below f_c ; and magnitude response 0 for all frequencies above f_c . The magnitude response can be plotted against the frequency to give Figure 1.

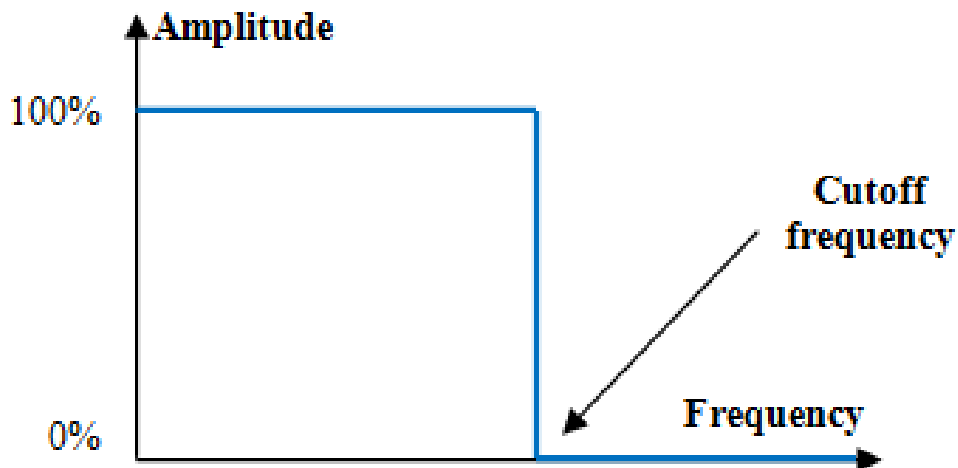


Figure 1: Magnitude transfer function for an ideal low-pass filter

RC circuits can act as low-pass or high-pass filters. If the voltage output is taken over the capacitor, then it will act as a low-pass filter; and if taken over the resistor, it will act as a high-pass filter (see Figures 2, 3).

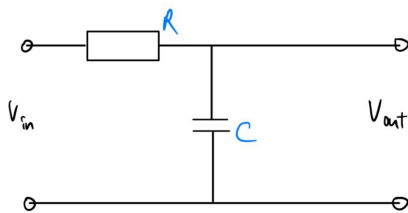


Figure 2: A RC circuit acting as a low-pass filter

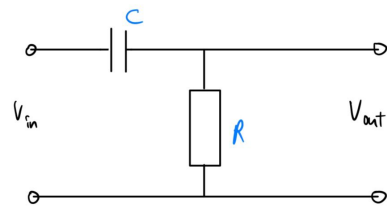


Figure 3: A RC circuit acting as a high-pass filter

1.2 Power Spectral Density (PSD) & White Noise

The *power spectral density* (PSD) is a function that describes how the power of the signal is distributed across various frequencies. In accordance with the physics and engineering convention, I will use the one-sided PSD, which means that only positive frequencies are included.

Gaussian white noise is characterized by having a constant PSD. Gaussian white noise can never be produced in reality, as that would imply that the signal has infinite power. So an upper limit on the frequency, i.e. a *bandwidth*, must be set. Let us call this bandwidth B (with units Hz).

The PSD of bandwidth-limited white noise (shown in Figure 4) is given by

$$S_{\text{input, noise}}(f) = \begin{cases} \frac{N_0}{2}, & \text{if } 0 \leq f \leq B \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

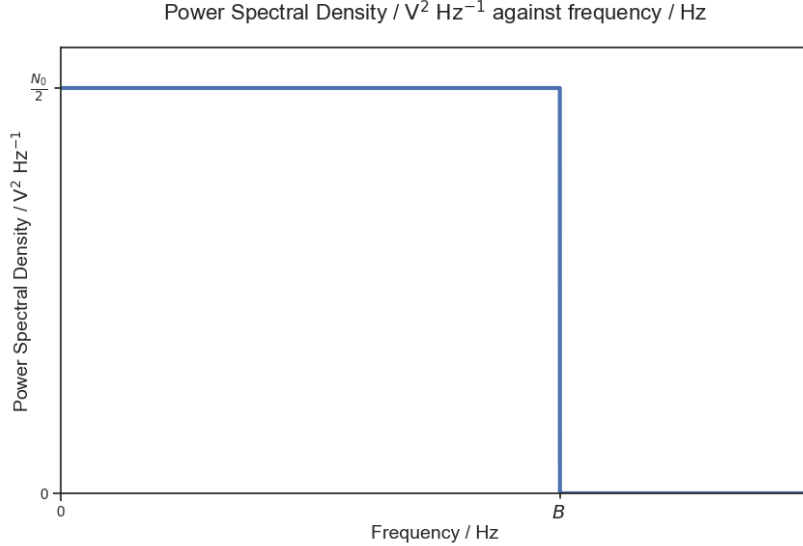


Figure 4: PSD of bandwidth-limited white noise

Here, N_0 is a constant known as the power spectral density. In signal processing, the convention is to assume that the signal is output over a 1Ω load, and make resistance dimensionless. This simplifies

$$P = \frac{V^2}{R} = V^2 \quad (2)$$

but leads to some confusing units. One would expect N_0 to carry units W Hz^{-1} , but since resistance is dimensionless, convention dictates N_0 carry units $\text{V}^2 \text{ Hz}^{-1}$ (see [3]). I will follow it, so that the dimensional analysis of my theoretical prediction works out.

1.3 Signal-to-Noise Ratio (SNR)

Noise in signals is generally undesirable, as it may obscure more important information in the signal. To quantify how much the noise is obscuring the signal, the *signal-to-noise ratio* (SNR) may be used. It is defined as

$$\text{SNR} = \frac{P_{\text{signal}}}{P_{\text{noise}}}, \quad (3)$$

where P_{signal} refers to the power of the desired signal, and P_{noise} refers to the power of the noise.

1.4 Motivation and Research Question

In general, noise reduction has many practical applications. For example, in daily life, noise reduction is applied in phone calls to improve call clarity and reduce background interference. Other applications include medical diagnostic tools like electrocardiograms and electroencephalograms, as well as in audio and image processing.

In this essay, I aim to explore the noise reduction effect of RC low-pass filters. More precisely, I would like to answer the question: *How does the frequency of an input sine wave f^* , with background white noise, affect the signal-to-noise ratio (SNR) after noise reduction through a RC low-pass filter?*

2 Qualitative Prediction

If the input sinusoidal signal has a high frequency, it will be removed by the low-pass filter, leading to weak signal power and thus low SNR. Conversely, if the input sinusoidal signal has a low frequency, it will pass through the low-pass filter, leading to strong signal power; and since high-frequency white noise will be removed, noise power will be reduced, leading to a high SNR.

3 Hypothesis

As the frequency f^* of the input sine wave increases, the SNR decreases.

4 Variables

Independent variable: Frequency of the input sine wave, f^* .

Dependent variable: SNR of the output signal of the RC low-pass filter.

Controlled variables:

1. Average power of white noise input.
2. Bandwidth of white noise input.
3. Amplitude of input sine wave.
4. Resistance in the RC circuit.
5. Capacitance in the RC circuit.
6. Fast Fourier Transform sampling rate.
7. Fast Fourier Transform frequency resolution.
8. Fast Fourier Transform window type (to be revisited in Section 11).

The Fast Fourier Transform will be used to measure the signal power in the output of the RC filter, so its settings need to be controlled across trials.

Uncontrolled variables:

1. Instantaneous power of white noise input.

Since white noise is inherently random, its power at a specific moment is unpredictable and uncontrollable.

5 Theoretical Model

5.1 Mathematical Tools

5.1.1 The Dirac Delta Function

I introduce the *Dirac delta function*, which will be a helpful mathematical tool in the next section. It is represented (see [4]) as follows:

$$\delta(x) = \begin{cases} 0, & \text{if } x \neq 0 \\ \infty, & \text{if } x = 0 \end{cases} \quad (4)$$

such that

$$\int_{-\infty}^{+\infty} \delta(x) dx = 1, \quad (5)$$

with the following useful property,

$$\int_{-\infty}^{+\infty} f(x) \delta(x - k) dx = f(k). \quad (6)$$

5.1.2 The Fourier Transform

The *Fourier Transform* (FT) is a tool that decomposes a signal into a sum of sinusoids. The FT accepts a continuous integrable function as an input, and outputs another function. The input function is in the *time domain*, which means it measures the instantaneous amplitude of the signal at a particular moment in time. The output function is in the *frequency domain*, which means it measures the strength of the component of the signal with a particular frequency.

Mathematically, the FT is defined by the integral

$$G(f) = \mathcal{F}\{g(t)\} = \int_{-\infty}^{\infty} g(t) e^{-j2\pi ft} dt \quad (7)$$

where $g(t)$ is the input function in the time domain, and $G(f)$ is the output function in the frequency domain (see [4]). Here j denotes the imaginary unit.

The FT can be helpful when solving differential equations. Consider what happens when a derivative $g'(t)$ is passed in:

$$\mathcal{F}\left\{\frac{dg(t)}{dt}\right\} = \int_{-\infty}^{\infty} \frac{dg(t)}{dt} e^{-j2\pi ft} dt \quad (8)$$

$$= [g(t)e^{-j2\pi ft}]_{-\infty}^{\infty} - \int_{-\infty}^{\infty} g(t) \cdot (-j2\pi f) e^{-j2\pi ft} dt \quad (9)$$

$$= 0 + j2\pi f \int_{-\infty}^{\infty} g(t) e^{-j2\pi ft} dt \quad (10)$$

$$= j2\pi f \mathcal{F}\{g\} \quad (11)$$

In (8), integration by parts was applied with $u = e^{-j2\pi ft}$, $du = (-j2\pi f) e^{-j2\pi ft} dt$; $v = g(t)$, $dv = \frac{dg(t)}{dt} dt$; in (9), the boundary terms vanish as, for any signal whose Fourier transform exists, $g(t) \rightarrow 0$ as $|t| \rightarrow \infty$ (see [4]).

The significance is that the differential equation in the time domain has been reduced into an algebraic equation in the frequency domain, which is much easier to work with.

5.2 Derivation of Theoretical Model

5.2.1 RC Low-pass Filters

We first derive the transfer function, defined in Section 1.1, for the RC low-pass filter, illustrated in Figure 5.

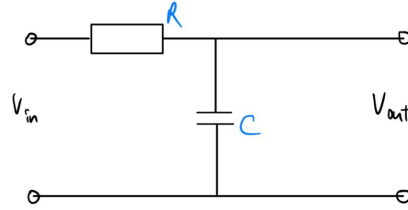


Figure 5: A RC low-pass filter (reproduction of Figure 2)

By Kirchhoff's voltage law,

$$v_{\text{in}}(t) = v_R(t) + v_C(t) \quad (12)$$

where $v_{\text{in}}(t)$ is the input voltage, and $v_R(t)$, $v_C(t)$ refer to the voltages on the resistor and capacitor respectively.

Write

$$v_{\text{in}}(t) = R \cdot i(t) \quad (13)$$

where $i(t)$ is the current, and

$$\begin{aligned} Q &= CV \\ \frac{dQ}{dt} &= \frac{d}{dt}(CV) \\ I &= C \frac{dV}{dt} \\ \therefore i(t) &= C \frac{dv_C(t)}{dt} \end{aligned} \quad (14)$$

Combining (12), (13), (14) gives

$$\begin{aligned} v_{\text{in}}(t) &= R \cdot i(t) + v_C(t) \\ &= RC \frac{dv_C(t)}{dt} + v_C(t) \end{aligned} \quad (15)$$

Since output voltage is taken over the capacitor (see Figure 5), $v_{\text{out}}(t) = v_C(t)$. Therefore

$$v_{\text{in}}(t) = RC \frac{dv_{\text{out}}(t)}{dt} + v_{\text{out}}(t). \quad (16)$$

Applying the Fourier transform to (16) gives

$$\begin{aligned} V_{\text{in}}(f) &= RC(-j2\pi f)V_{\text{out}}(f) + V_{\text{out}}(f) \quad (\text{by 11}) \\ &= V_{\text{out}}(f)(1 + j2\pi fRC) \\ \frac{V_{\text{out}}(f)}{V_{\text{in}}(f)} &= \frac{1}{1 + j2\pi fRC} \end{aligned} \quad (17)$$

The transfer function of the RC filter is then given by

$$|H(f)| = \left| \frac{V_{\text{out}}(f)}{V_{\text{in}}(f)} \right| = \left| \frac{1}{1 + j2\pi fRC} \right| = \frac{1}{\sqrt{1 + (2\pi fRC)^2}}. \quad (18)$$

Note that when deriving (18), no assumptions were made about the frequency components of the signal; i.e. the signal could contain noise, and (18) would still hold for each frequency.

For a RC low-pass filter, the cutoff frequency f_C is defined as frequency at which the filter attenuates the signal to half its unfiltered power (see [5]). This means amplitude is $\frac{1}{\sqrt{2}}$ of the original, i.e.

$$|H(f_C)| = \frac{1}{\sqrt{2}} \quad (19)$$

$$\frac{1}{\sqrt{1 + (2\pi f_C RC)^2}} = \frac{1}{\sqrt{2}} \quad (20)$$

$$1 + (2\pi f_C RC)^2 = 2 \quad (21)$$

$$2\pi f_C RC = 1 \quad (22)$$

$$f_C = \frac{1}{2\pi RC} \quad (23)$$

The transfer function and cutoff frequency are shown in Figure 6.

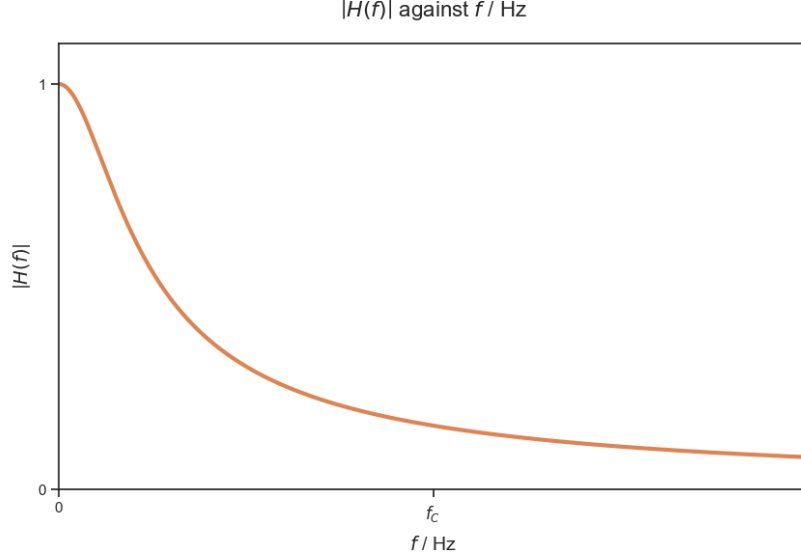


Figure 6: Transfer function of a RC low-pass filter

5.2.2 SNR Calculation

We want to compute the signal-to-noise ratio of the output signal from the RC low-pass filter. Recalling that

$$\text{SNR} = \frac{P_{\text{output, signal}}}{P_{\text{output, noise}}}, \quad (24)$$

we will first tackle the denominator, and find the power of the output signal when only bandwidth-limited white noise is input into the RC filter.

Its PSD is given by

$$S_{\text{input, noise}}(f) = \begin{cases} \frac{N_0}{2}, & \text{if } 0 \leq f \leq B \\ 0, & \text{otherwise} \end{cases} \quad (25)$$

Using (18), the power of the inputs and outputs is related by:

$$\frac{P_{\text{output}}(f)}{P_{\text{input}}(f)} = \left(\frac{V_{\text{output}}(f)}{V_{\text{input}}(f)} \right)^2 = |H(f)|^2 = \frac{1}{1 + (2\pi f RC)^2} \quad (26)$$

Applying (26) for every frequency in the PSD gives us:

$$S_{\text{output, noise}}(f) = S_{\text{input, noise}}(f) \cdot |H(f)|^2 \quad (27)$$

To find the total power, we integrate (27) to get:

$$P_{\text{output, noise}} = \int_0^\infty S_{\text{input, noise}}(f) \cdot |H(f)|^2 df \quad (28)$$

$$= \frac{N_0}{2} \int_0^B \frac{1}{1 + (2\pi f RC)^2} df \quad (29)$$

$$= \frac{N_0 \arctan(2\pi BRC)}{4\pi RC} \quad (30)$$

Now we move on to find $P_{\text{output, signal}}$, i.e. the power of the output signal when bandwidth-limited white noise and a pure sine wave are input into the RC filter together. Suppose that the pure sine wave has frequency f^* and peak voltage amplitude V_0 . This means its r.m.s. amplitude is $\frac{V_0}{\sqrt{2}}$, so its power spectral density is

$$\begin{cases} \frac{V_0^2}{2}, & \text{if } f = f^* \\ 0, & \text{otherwise} \end{cases} = \frac{V_0^2}{2} \delta(f - f^*). \quad (31)$$

Then the new PSD is given by

$$S_{\text{input, signal}}(f) = \frac{V_0^2}{2} \delta(f - f^*) \quad (32)$$

The PSD for the output signal is given by

$$S_{\text{output, signal}}(f) = S_{\text{input, signal}}(f) \cdot |H(f)|^2 \quad (33)$$

To find the total power, we integrate (33) to get:

$$P_{\text{output, signal}} = \int_0^\infty S_{\text{input, signal}}(f) \cdot |H(f)|^2 df \quad (34)$$

$$= \int_0^\infty \frac{V_0^2}{2} \delta(f - f^*) \cdot |H(f)|^2 df \quad (35)$$

$$= \frac{V_0^2}{2} \int_0^\infty \delta(f - f^*) \cdot |H(f)|^2 df \quad (36)$$

$$= \frac{V_0^2}{2} |H(f^*)|^2 \quad (\text{by (6)}) \quad (37)$$

$$= \frac{V_0^2}{2(1 + (2\pi f^* RC)^2)} \quad (38)$$

Apply (30) and (38):

$$\text{SNR} = \frac{P_{\text{output, signal}}}{P_{\text{output, noise}}} = \frac{\frac{V_0^2}{2(1 + (2\pi f^* RC)^2)}}{\frac{N_0 \arctan(2\pi BRC)}{4\pi RC}} = \frac{2\pi RC V_0^2}{N_0 \arctan(2\pi BRC) (1 + (2\pi f^* RC)^2)} \quad (39)$$

5.3 Remarks on the Theoretical Model

(39) addresses the research question. As f^* increases, SNR decreases, confirming our qualitative prediction in Section 2.

Let us examine the units. SNR is a ratio and thus dimensionless. Performing dimensional analysis on

$\frac{2\pi RC V_0^2}{N_0 \arctan(2\pi BRC)(1+(2\pi f^* RC)^2)}$ gives

$$\frac{\Omega \times F \times V^2}{V^2 \text{ Hz}^{-1}} = \frac{s \times V^2}{V^2 \times s} = 1. \quad (40)$$

Indeed, the convention that N_0 carries units $V^2 \text{ Hz}^{-1}$ is necessary for the validity of (39).

6 Methodology

6.1 Fast Fourier Transform and Window Functions

The Fast Fourier Transform (FFT) will be vital for measuring the signal power. To prepare for the methodology, the mathematical ideas related to the FFT are introduced here.

6.1.1 The Discrete Fourier Transform and Fast Fourier Transform

In the real world, we usually deal with sampled signals consisting of discrete data points, rather than continuous functions. The FT only accepts continuous functions, so a discrete analogue of the FT is needed.

The *Discrete Fourier Transform* (DFT) is precisely that. Mathematically, it is given by

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi j \frac{k}{N} n} \quad (41)$$

where x_k are the input data samples, and X_k are the frequency bin coefficients, i.e. a complex number representing the phase and amplitude of a specific frequency component present in the input signal.

The *Fast Fourier Transform* (FFT) is an algorithm to compute the DFT highly efficiently. It is available on many oscilloscopes for analyzing input signals.

6.1.2 Spectral Leakage, Windowing, and Correction Factors

Suppose a pure sine wave is input into the FFT. Ideally, all its amplitude should be contained in a single frequency bin. However, in practice, if a sine wave is not right in the middle of a frequency bin, a phenomenon known as *spectral leakage* occurs, where the amplitude of the tone is distributed across neighbouring frequency bins, instead of in a single frequency bin. Spectral leakage in the FFT cannot be removed entirely, but there are ways to mitigate its effects.

To address the problem, the frame of input data can be multiplied by a *window function* before performing the FFT. The purpose is to taper the signal to zero at the edges of the frame, which eliminates the sharp discontinuity that causes leakage. However, there is always a trade-off between the *frequency resolution* and the *dynamic range*.

The frequency resolution refers to the ability to distinguish two components with frequencies very close to each other (see [6]). This can be measured with the mainlobe width, which is the width of the central peak in the FFT frequency domain graph. A wider mainlobe smears the amplitude of a component over more adjacent frequency bins, causing a lower frequency resolution.

The dynamic range refers to the span of small-to-large frequencies that can be properly measured without large distortion by the FFT (see [6]). This can be measured by the sidelobe level, which is the height of the side peaks in the FFT frequency domain graph. Higher sidelobes mean the leakage from a strong signal can be seen

far away in the spectrum, making it hard to see a weak signal next to a strong one, causing a lower dynamic range.

Various windows for the FFT exist, with all of them having a tradeoff between narrowing the mainlobe and suppressing the sidelobes.

Figures 7 to 10 (taken from [7]) illustrate four common FFT windows, namely the Rectangle, Hanning, Hamming and Blackman windows. Table 1 is a brief comparison of the four windows.

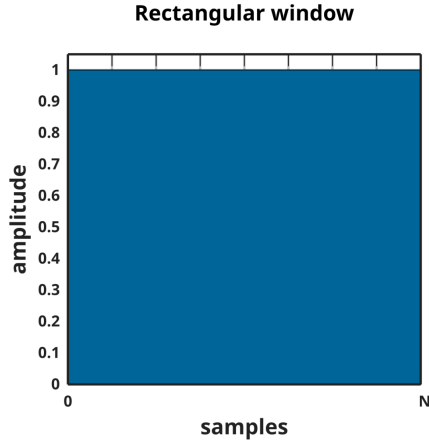


Figure 7: Rectangle window

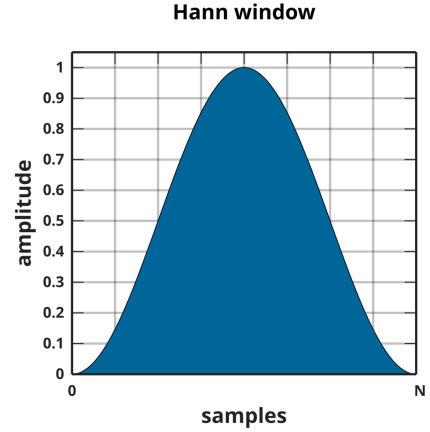


Figure 8: Hanning window

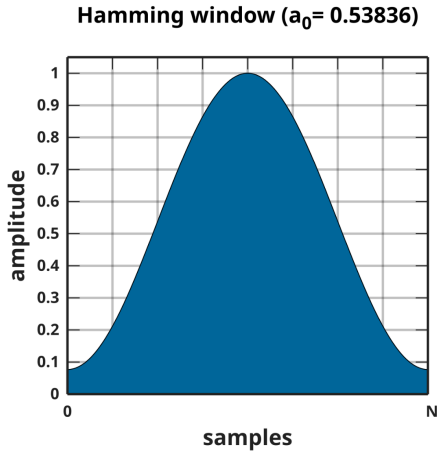


Figure 9: Hamming window

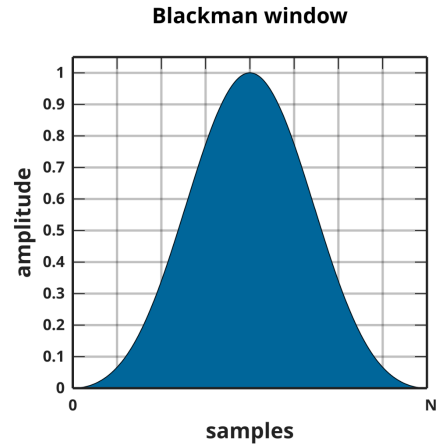


Figure 10: Blackman window

Window type	Trade-off
Rectangular	Narrowest mainlobe, worst sidelobes
Hanning (a.k.a. Hann)	Good balance between mainlobe width and sidelobe suppression
Hamming	Nearest sidelobe more suppressed than Hanning, but far out sidelobes less suppressed
Blackman	Wider mainlobe but lower sidelobes

Table 1: Qualitative comparison of various FFT windows (see [6])

When applying a window function to a signal before FFT, the amplitude and energy of the signal is reduced.

To correct for this and preserve energy across frequency bins, an energy correction factor corresponding to the window function has to be multiplied to the recorded voltage measurements (see [8]), shown in Table 2.

Window type	Energy correction factor
Rectangular	1.00
Hanning	1.63
Hamming	1.59
Blackman	1.97

Table 2: Energy correction factors for various FFT windows

In general, the Hanning window is the most widely used, and it is excellent for most spectrum analysis purposes (see [6]), so it will be used in Section 6.2.

6.2 Methodology

To carry out my experiment, I need a source of white noise.

6.2.1 Method 1: White Noise Generation with Transistors

My first attempt was to build my own white noise generator. The circuit shown in Figure 11, from [9], makes use of transistors to generate white noise.

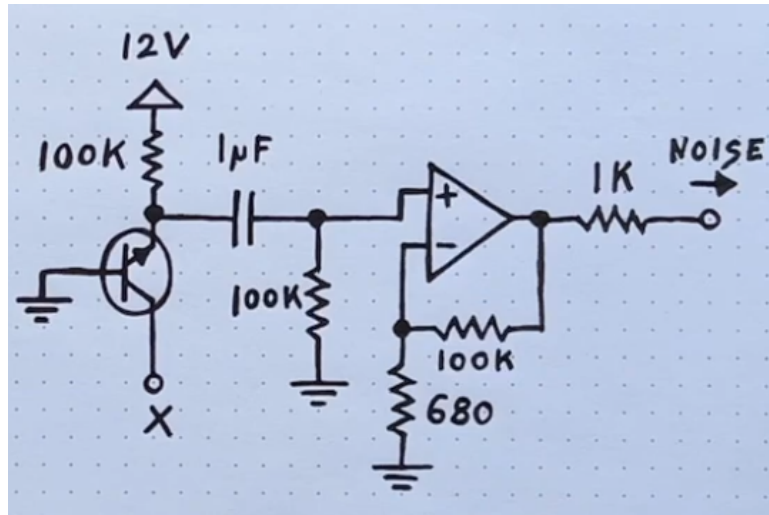


Figure 11: Circuit diagram of white noise generator built with transistors

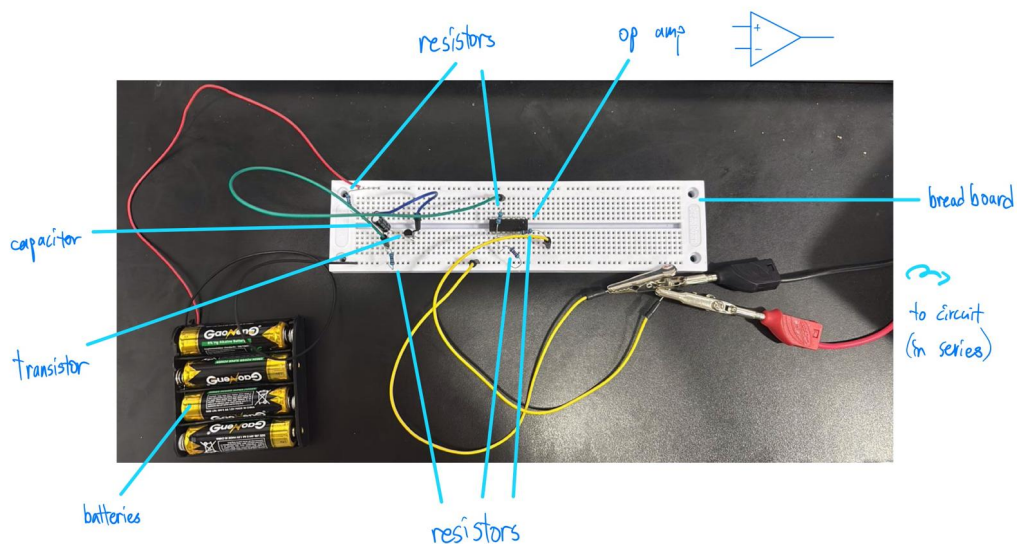


Figure 12: White noise generator built with transistors

I built this circuit, shown in Figure 12. However, I encountered a problem. The power spectral density

was uneven. When I put the signal into a FFT, lower frequencies had a much higher amplitude than higher frequencies (see Figure 13), so the noise was not white, and hence unsuitable for the experiment.

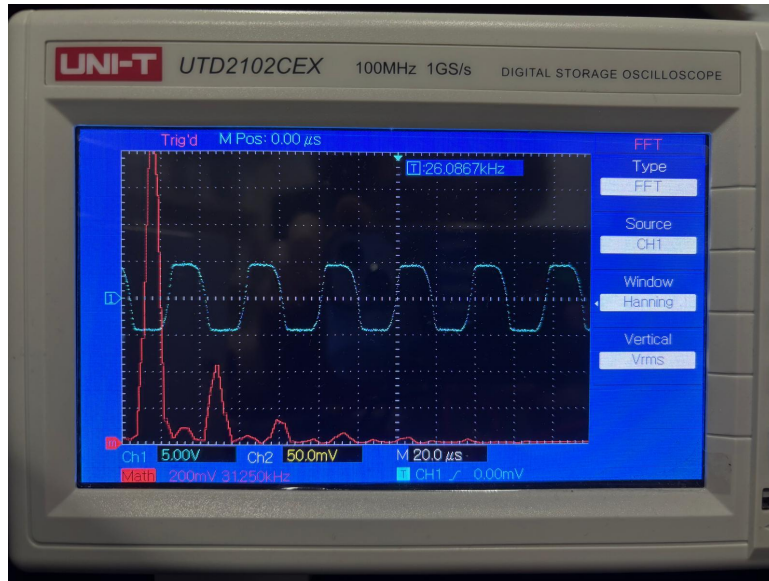


Figure 13: Blue graph: Waveform of noise generated by transistor (V_{RMS} against t)
Red graph: FFT of noise generated by transistor (V_{RMS} against f)

I tried rebuilding the circuit, but this did not fix the problem, so I rejected this method.

6.2.2 Method 2: Computer White Noise Generation

To execute this method, I used an online website, mynoise.net [10], to generate white noise on my computer. I recorded some of the noise generated by the website into a .mp3 file, then analysed the audio spectrum.

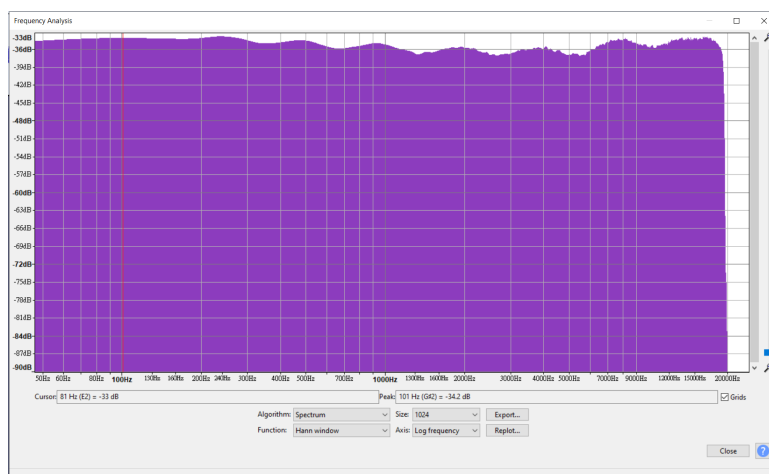


Figure 14: Audio spectrum analysis (FFT) by Audacity

Using Audacity, an audio editor software, I performed a FFT on the .mp3 file. The FFT plot is given in Figure 14, from which it is clear that the amplitude (and hence power) is constant across all frequencies between

0 to 20000 Hz, so this noise is indeed white noise. This also constitutes the measurement of B , as the bandwidth can be found by looking at the maximum supported frequency, 20000 Hz.

After obtaining a .mp3 file of white noise, I needed to input this into the circuit. To this end, I cut open some earphones to reveal the wires inside (see Figure 15). By plugging in the earphone jack into the computer, I could turn the audio output of my computer into electronic signals. The voltage across the earphones was rather small, so I input it into an amplifier (see Figure 16). This was the white noise signal passed into my circuit.

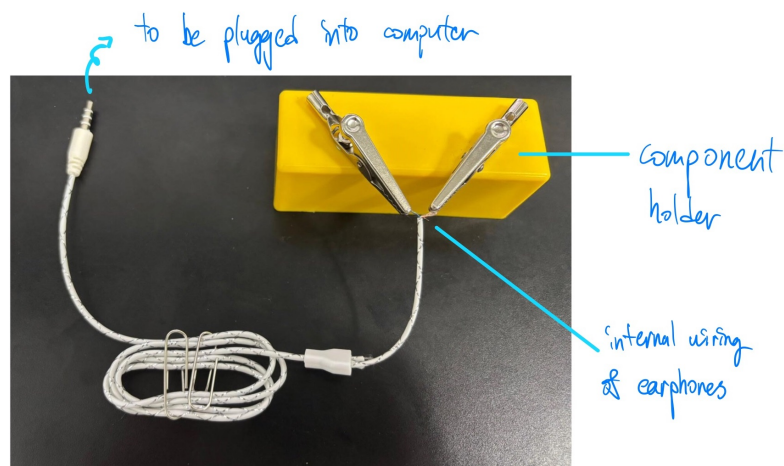


Figure 15: Earphones connected to a component holder

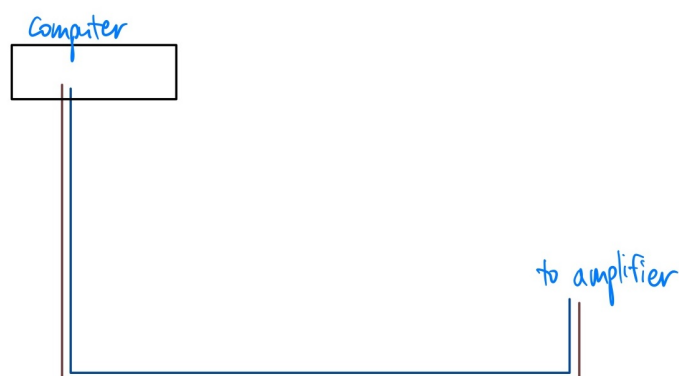


Figure 16: Circuit diagram of cut earphones

6.2.3 Pre-FFT Data Processing

To find the SNRs experimentally, I need to measure $V_{\text{output, signal}}$ and $V_{\text{output, noise}}$. The latter is a constant, and its measurement will be addressed in Section 8.1. As for the former, I need to measure the r.m.s. amplitude of a particular frequency component in a noisy signal, hence the need for the FFT.

The FFT amplitude reading at the frequency component f^* does not actually represent $V_{\text{output, signal}}$. This is because the noise has some amplitude at f^* , so the FFT amplitude reading measures the noise and signal amplitudes at f^* .

To account for this, I used the data acquisition mode of the oscilloscope to be "Average: 256 times". This means that the oscilloscope averages the readings in the time domain over 256 trials. This was intended to allow the noise signals to cancel each other out in the time domain, leaving the pure sine wave component, which could then be used in the FFT.

6.3 Apparatus

1. Cut earphones (as described above), x1
2. Amplifier (ungrounded), x1
3. Signal generator (ungrounded), x1
4. Oscilloscope, with FFT functionality, x1
5. Resistor, x1
6. Capacitor, x1
7. Wires, crocodile clips and component holders

6.4 Experimental Set-up

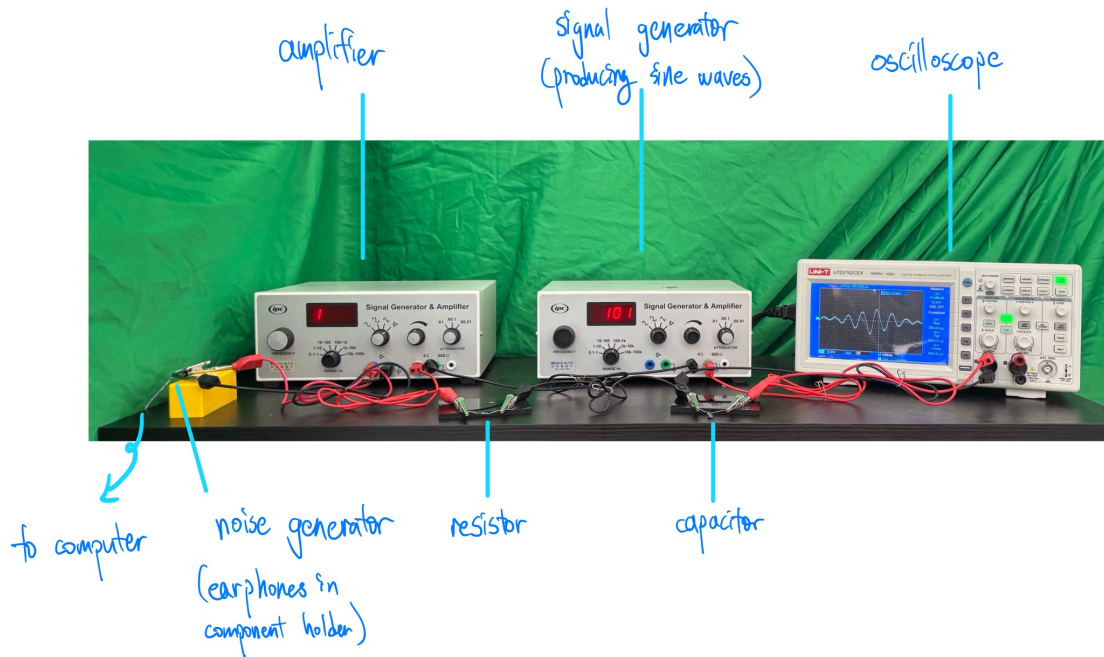


Figure 17: Experimental set-up

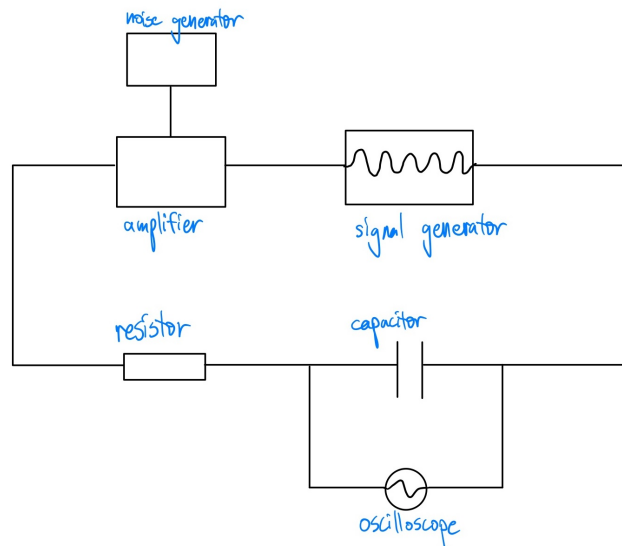


Figure 18: Circuit diagram

My experimental set-up is shown in Figure 17, which is equivalent to the circuit diagram in Figure 18.

The amplifier and signal generator both had ground connections. To connect the two devices in series, I had to connect the outputs of the amplifier and signal generator, which would have grounded the whole circuit and

removed the signal. So, I needed to unground both the amplifier and the signal generator. After consulting with my supervisor regarding safety measures, I accomplished this by physically removing the ground pin in the power plugs of both (see Figure 19).

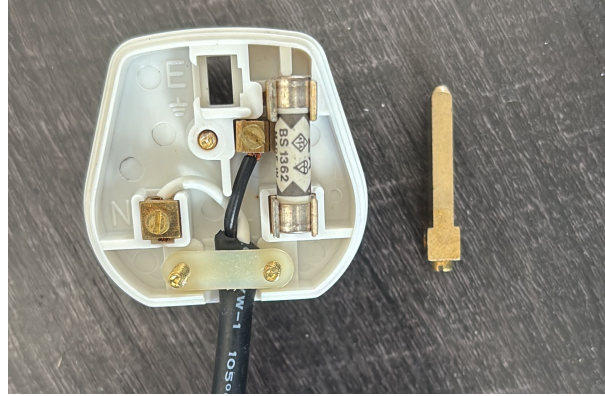


Figure 19: Ungrounded power plug

7 Experimental Procedures

1. Set the data acquisition mode of the oscilloscope to "Average: 256 times".
2. Set the output frequency of the sine wave signal generator to 100 Hz.
3. Wait for the FFT amplitude reading at 100 Hz to stabilize (as much as possible).
4. Record the amplitude reading.
5. Repeat Steps 2-4 for frequencies $f^* = 200, 300, \dots 1000$ Hz.
6. Repeat Steps 2-5 four times for a total of 5 trials.

8 Data Treatment

8.1 Measuring and Defining Constants

B denotes the bandwidth of the bandwidth-limited white noise. As measured in Section 6, $B = 20000$ Hz.

V_0 is the peak voltage amplitude of the sine wave in the signal. The choice of V_0 is somewhat arbitrary. I chose $V_0 = 6.00 \pm 0.01$ V, which my signal generator is indeed capable of. To measure the value, I input the pure sine wave signal into the oscilloscope, and read off the peak-to-peak voltage.

R is the resistance of the resistor. By direct measurement with a multimeter, the resistor has resistance $R = 330 \pm 1$ Ω . However, the signal generator also has resistance 4.0 ± 0.1 Ω , so the value that should be used in the calculation is $R = 334 \pm 1$ Ω .

C , the capacitance of the capacitor, was similarly found to be $C = 1.00 \pm 0.01$ μF .

To measure $V_{\text{RMS, output, noise}}$, I input pure noise into the RC filter, then measured the voltage across the capacitor with an oscilloscope to obtain $V_{\text{RMS, output, noise}} = 5.40 \pm 0.01$ V. I also input pure noise (without passing through the RC filter) into the oscilloscope directly to obtain $V_{\text{RMS, input, noise}} = 6.34 \pm 0.01$ V and $N_0 = 10.04 \pm 0.01$ $\text{V}^2 \text{ Hz}^{-1}$.

Here is a summary:

Constant	Value
B	20000 Hz
V_0	6.00 ± 0.01 V
R	334 ± 1 Ω
C	$(1.00 \pm 0.01) \times 10^{-6}$ F
$V_{\text{RMS, output, noise}}$	5.40 ± 0.01 V
$V_{\text{RMS, input, noise}}$	6.34 ± 0.01 V
N_0	10.04 ± 0.01 $\text{V}^2 \text{ Hz}^{-1}$

Table 3: Measured constants

8.2 Raw Data

The raw data is displayed in Table 4.

The $V_{\text{output, signal}}$ is an average taken over the five trials. I chose the half-range method to find uncertainties as opposed to standard deviation, as the sample size is too small for the sample standard deviation to be a reliable estimator of the underlying uncertainty. The frequencies were read off from the signal generator, with an absolute uncertainty of 1 Hz. A sample calculation is shown below, for $f^* = 100$ Hz:

f^* / Hz	Raw $V_{\text{output, signal}} / \text{V}$ ($\pm 0.001\text{V}$)					Raw $\bar{V}_{\text{output, signal}} / \text{V}$
	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	
100 ± 1	0.485	0.510	0.470	0.460	0.440	0.47 ± 0.04
200 ± 1	0.350	0.410	0.390	0.360	0.405	0.38 ± 0.03
300 ± 1	0.275	0.270	0.260	0.305	0.250	0.27 ± 0.03
400 ± 1	0.255	0.235	0.275	0.255	0.240	0.25 ± 0.02
500 ± 1	0.230	0.235	0.245	0.195	0.220	0.23 ± 0.03
600 ± 1	0.230	0.175	0.220	0.190	0.185	0.20 ± 0.03
700 ± 1	0.190	0.180	0.195	0.220	0.225	0.20 ± 0.02
800 ± 1	0.160	0.160	0.180	0.145	0.150	0.16 ± 0.02
900 ± 1	0.140	0.140	0.125	0.130	0.125	0.13 ± 0.01
1000 ± 1	0.090	0.110	0.105	0.090	0.120	0.10 ± 0.02

Table 4: Raw data table

$$V_{\text{output, signal}} = \frac{0.485 + 0.510 + 0.470 + 0.460 + 0.440}{5} \text{V} = 0.473\text{V}$$

$$\begin{aligned} \Delta V_{\text{output, signal}} &= \frac{\max - \min}{2} = \frac{0.510 - 0.440}{2} \text{V} = 0.035\text{V} \\ &= 0.04\text{V} \text{ (1 s.f.)} \end{aligned}$$

$$\therefore V_{\text{output, signal}} = 0.47 \pm 0.04\text{V}$$

8.3 Processed Data

f^* / Hz	Raw $\bar{V}_{\text{output, signal}} / \text{V}$	Corrected $\bar{V}_{\text{output, signal}} / \text{V}$	SNR / 10^{-3}
100 ± 1	0.47 ± 0.04	0.77 ± 0.07	10 ± 2
200 ± 1	0.38 ± 0.03	0.62 ± 0.05	7 ± 1
300 ± 1	0.27 ± 0.03	0.44 ± 0.05	3.3 ± 0.8
400 ± 1	0.25 ± 0.02	0.41 ± 0.03	2.9 ± 0.4
500 ± 1	0.23 ± 0.03	0.37 ± 0.05	2.3 ± 0.6
600 ± 1	0.20 ± 0.03	0.33 ± 0.05	1.9 ± 0.6
700 ± 1	0.18 ± 0.03	0.29 ± 0.05	1.4 ± 0.5
800 ± 1	0.16 ± 0.02	0.26 ± 0.03	1.2 ± 0.3
900 ± 1	0.13 ± 0.01	0.21 ± 0.02	0.8 ± 0.1
1000 ± 1	0.10 ± 0.02	0.16 ± 0.03	0.4 ± 0.2

Table 5: Processed data table

Recalling Section 6.1.2 and applying the correction factor for the Hanning window,

$$\text{Corrected } \bar{V}_{\text{output, signal}} = \text{Raw } \bar{V}_{\text{output, signal}} \times 1.63 \quad (42)$$

The conversion between RMS and peak-to-peak voltage is given by

$$V_{\text{RMS, output, signal}} = \frac{V_{\text{output, signal}}}{\sqrt{2}}, \quad (43)$$

so the SNRs are found by applying

$$\text{SNR} = \left(\frac{V_{\text{RMS, output, signal}}}{V_{\text{RMS, output, noise}}} \right)^2 = \frac{1}{2} \left(\frac{V_{\text{output, signal}}}{V_{\text{RMS, output, noise}}} \right)^2. \quad (44)$$

A sample calculation is shown below, for $f^* = 100$ Hz. The correction factor is applied:

$$\text{Corrected } V_{\text{output, signal}} = \text{Raw } V_{\text{output, signal}} \times 1.63 = 0.47 \text{ V} \times 1.63 = 0.7661 \text{ V}$$

$$\begin{aligned} \Delta \text{Corrected } V_{\text{output, signal}} &= \Delta \text{Raw } V_{\text{output, signal}} \times 1.63 = 0.04 \text{ V} \times 1.63 = 0.0652 \text{ V} \\ &= 0.07 \text{ V (1 s.f.)} \end{aligned}$$

$$\therefore \text{Corrected } V_{\text{output, signal}} = 0.77 \pm 0.07 \text{ V}$$

Then SNR is calculated:

$$\text{SNR} = \frac{1}{2} \left(\frac{0.77}{5.40} \right)^2 = 0.010166$$

$$\begin{aligned} \frac{\Delta \text{SNR}}{\text{SNR}} &= 2 \times \left(\frac{\Delta V_{\text{output, signal}}}{V_{\text{output, signal}}} + \frac{\Delta V_{\text{RMS, output, noise}}}{V_{\text{RMS, output, noise}}} \right) \\ \Delta \text{SNR} &= 2 \times \left(\frac{0.07}{0.77} + \frac{0.01}{5.40} \right) \times 0.010166 = 0.00189 \\ &= 0.002 \text{ (1 s.f.)} \end{aligned}$$

$$\therefore \text{SNR} = 0.010 \pm 0.002$$

9 Discussion

9.1 Mathematical Analysis

By (39),

$$\text{SNR} = \frac{2\pi R C V_0^2}{N_0 \arctan(2\pi B R C) \left(1 + (2\pi f^* R C)^2\right)}, \quad (45)$$

where $R, C, V_0, B, N_0, P_{\text{output, signal}}$ are constants. This gives us the relationship

$$\text{SNR} \propto \left(1 + (2\pi f^* R C)^2\right)^{-1}. \quad (46)$$

Note that both SNR and $\left(1 + (2\pi f^* R C)^2\right)^{-1}$ are dimensionless quantities. Performing dimensional analysis on $2\pi f^* R C$, its units are given by

$$\text{Hz} \cdot \Omega \cdot \text{F} = \frac{1}{\text{s}} \cdot \frac{\text{V}}{\text{A}} \cdot \frac{\text{A} \cdot \text{s}}{\text{V}} = 1.$$

Since R, C were already measured in Section 8.1, it is straightforward to compute $\left(1 + (2\pi f^* R C)^2\right)^{-1}$ (see Table 6).

f^* / Hz	$\left(1 + (2\pi f^* R C)^2\right)^{-1}$
100 ± 1	0.958 ± 0.002
200 ± 1	0.850 ± 0.005
300 ± 1	0.716 ± 0.007
400 ± 1	0.587 ± 0.008
500 ± 1	0.476 ± 0.007
600 ± 1	0.387 ± 0.007
700 ± 1	0.317 ± 0.006
800 ± 1	0.262 ± 0.006
900 ± 1	0.219 ± 0.005
1000 ± 1	0.185 ± 0.004

Table 6: Calculation of $\left(1 + (2\pi f^* R C)^2\right)^{-1}$.

The calculation of $\left(1 + (2\pi f^* R C)^2\right)^{-1}$ with uncertainties is shown below, for $f^* = 100 \text{ Hz}$. First, find $(2\pi f^* R C)^2$ and its uncertainty.

$$(2\pi f^* R C)^2 = (2\pi \times 100 \times 334 \times 1.00 \times 10^{-6})^2 = 0.044041$$

$$\begin{aligned} \frac{\Delta(2\pi f^* R C)^2}{(2\pi f^* R C)^2} &= 2 \times \left(\frac{\Delta f^*}{f^*} + \frac{\Delta R}{R} + \frac{\Delta C}{C} \right) \\ \Delta(2\pi f^* R C)^2 &= 2 \times \left(\frac{1}{100} + \frac{1}{334} + \frac{0.01 \times 10^{-6}}{1.00 \times 10^{-6}} \right) \times 0.044041 = 0.002025 \end{aligned}$$

When finding $\left(1 + (2\pi f^* RC)^2\right)$, note that 1 carries no uncertainty, so

$$\Delta \left(1 + (2\pi f^* RC)^2\right) = \Delta (2\pi f^* RC)^2 = 0.002025$$

Now to find $\Delta \left(1 + (2\pi f^* RC)^2\right)^{-1}$:

$$\left(1 + (2\pi f^* RC)^2\right)^{-1} = \frac{1}{1 + 0.044041} = 0.9578$$

$$\begin{aligned} \frac{\Delta \left(1 + (2\pi f^* RC)^2\right)^{-1}}{\left(1 + (2\pi f^* RC)^2\right)^{-1}} &= \frac{\Delta \left(1 + (2\pi f^* RC)^2\right)}{\left(1 + (2\pi f^* RC)^2\right)} \\ \frac{\Delta \left(1 + (2\pi f^* RC)^2\right)^{-1}}{0.9578} &= \frac{0.002025}{1 + 0.044041} \\ \Delta \left(1 + (2\pi f^* RC)^2\right)^{-1} &= 0.001858 \\ &= 0.002 \text{ (1 s.f.)} \end{aligned}$$

9.2 Graphical Analysis

Motivated by the directly proportional relationship of SNR and $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ (see (46)), SNR against $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ is plotted in Figure 20, using values calculated in Sections 8.3 and 9.1.

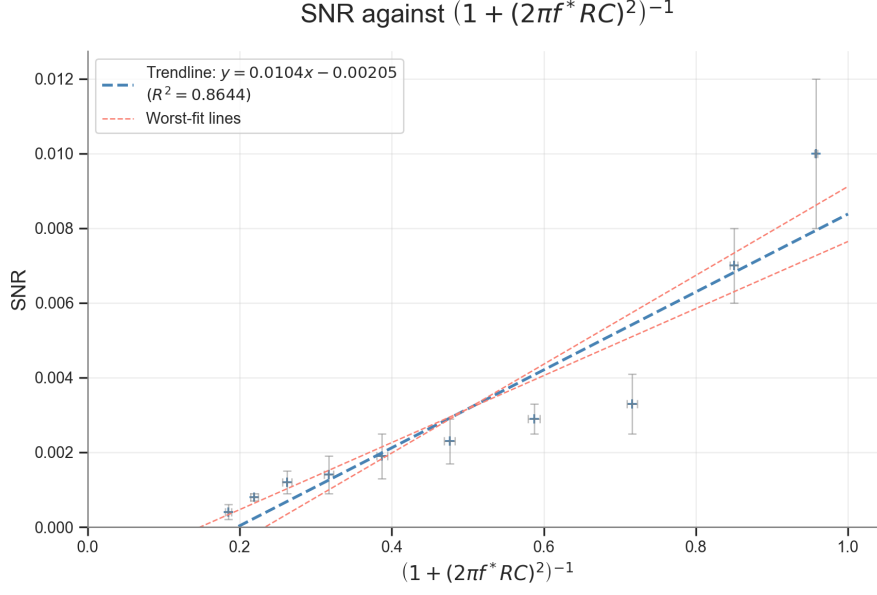


Figure 20: SNR against $\left(1 + (2\pi f^* RC)^2\right)^{-1}$, with $f^* = 100, 200, \dots, 1000$ Hz

The best- and worst-fit lines were obtained with Excel LINEST. It is quite clear that there are significant errors present, leading to a very low R^2 value of 0.8644. However, it seems that the eight data points on the left have a linear relationship, while only the two rightmost data points deviate from the linear trend significantly, to be discussed in Section 10. For now, SNR against $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ will be plotted again, but this time discarding $f^* = 100, 200$ Hz (see Figure 21).

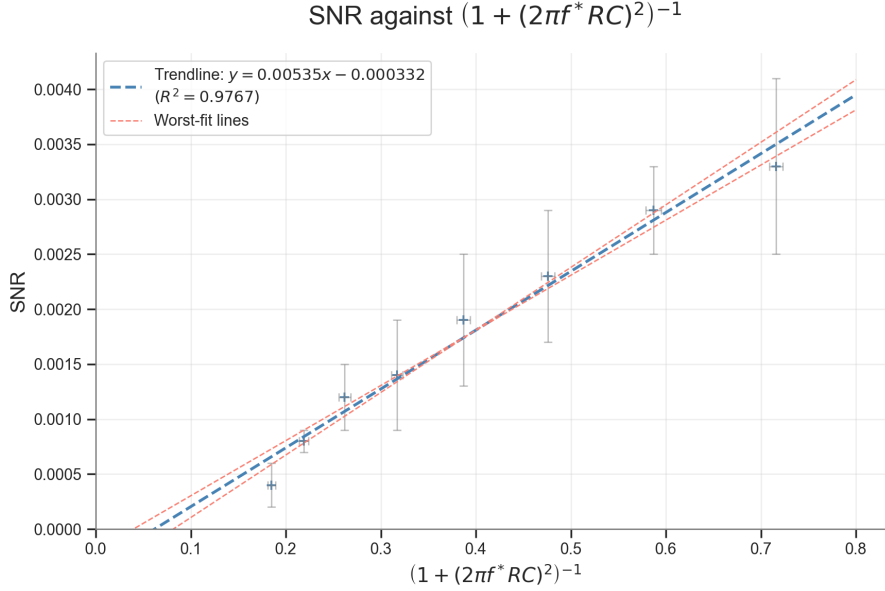


Figure 21: SNR against $\left(1 + (2\pi f^* RC)^2\right)^{-1}$, with $f^* = 300, 400, \dots, 1000$ Hz

Figure 21 indeed has a much higher R^2 value of 0.9767. Graphically, the best-fit line passes through all the error bars, indicating good agreement between the theory and experimental data. From Excel LINEST, the

slope m and y-intercept c are found to be $m = 0.0054 \pm 0.0003$, $c = -0.0003 \pm 0.0001$. Section 5.2 finds SNR to be directly proportional to $\left(1 + (2\pi f^* RC)^2\right)^{-1}$, predicting a y-intercept of zero. The small but non-zero magnitude of the y-intercept suggests the presence of systematic errors, to be addressed in Section 10.

9.3 Comparison Between Experimental and Theoretical Relationship

We apply (39) and use the measured values of R, C in Section 8.1 and calculated values of $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ in Section 9.1 to find the theoretical SNRs (see Table 7).

f^* / Hz	$\left(1 + (2\pi f^* RC)^2\right)^{-1}$	Theoretical SNR / 10^{-3}
100 ± 1	0.958 ± 0.002	4.66 ± 0.09
200 ± 1	0.850 ± 0.005	4.13 ± 0.03
300 ± 1	0.716 ± 0.007	3.48 ± 0.03
400 ± 1	0.587 ± 0.008	2.86 ± 0.04
500 ± 1	0.476 ± 0.007	2.32 ± 0.03
600 ± 1	0.387 ± 0.007	1.88 ± 0.03
700 ± 1	0.317 ± 0.006	1.54 ± 0.03
800 ± 1	0.262 ± 0.006	1.27 ± 0.03
900 ± 1	0.219 ± 0.005	1.07 ± 0.02
1000 ± 1	0.185 ± 0.004	0.90 ± 0.02

Table 7: Theoretical SNR calculation

An example computation for $f^* = 300$ is given below. First, find $\arctan(2\pi BRC)$ with uncertainties:

$$\arctan(2\pi BRC) = \arctan(2\pi \times 20000 \times 334 \times 1.00 \times 10^{-6}) = 1.5469752$$

To compute the uncertainty, we need to use the half-range method on $\arctan(2\pi BRC)$. Recall $R = 334 \pm 1\Omega$, so $R_+ = 335\Omega$ and $R_- = 333\Omega$; similarly $C = (1.00 \pm 0.01) \times 10^{-6}\text{F}$, so $C_+ = 1.01 \times 10^{-6}\text{F}$ and $C_- = 0.99 \times 10^{-6}\text{F}$.

$$\begin{aligned}
\Delta \arctan(2\pi BRC) &= \frac{\max - \min}{2} \\
&= \frac{\arctan(2\pi BR_+C_+) - \arctan(2\pi BR_-C_-)}{2} \\
&= \frac{\arctan(2\pi \times 20000 \times 335 \times 1.01 \times 10^{-6}) - \arctan(2\pi \times 20000 \times 333 \times 0.99 \times 10^{-6})}{2} \\
&= 0.000309448
\end{aligned}$$

Now compute the theoretical SNR:

$$\begin{aligned}
\text{Theoretical SNR} &= \frac{2\pi R C V_0^2}{N_0 \arctan(2\pi B R C) \left(1 + (2\pi f^* R C)^2\right)} = \frac{2\pi R C V_0^2}{N_0 \arctan(2\pi B R C)} \left(1 + (2\pi f^* R C)^2\right)^{-1} \\
&= \frac{2\pi \times 334 \times 1.00 \times 10^{-6} \times 6.00^2}{10.04 \times 1.5469752} \times 0.716 \\
&= 4.6599 \times 10^{-3}
\end{aligned}$$

Then the uncertainty can be found as follows:

$$\begin{aligned}
\frac{\Delta \text{Theoretical SNR}}{\text{Theoretical SNR}} &= \frac{\Delta N_0}{N_0} + \frac{\Delta R}{R} + \frac{\Delta C}{C} + 2 \frac{\Delta V_0}{V_0} + \frac{\Delta \arctan(2\pi B R C)}{\arctan(2\pi B R C)} + \frac{\Delta \left(1 + (2\pi f^* R C)^2\right)^{-1}}{\left(1 + (2\pi f^* R C)^2\right)^{-1}} \\
\frac{\Delta \text{Theoretical SNR}}{4.6599 \times 10^{-3}} &= \frac{0.01}{10.04} + \frac{1}{334} + \frac{0.01 \times 10^{-6}}{1.00 \times 10^{-6}} + 2 \times \frac{0.01}{6.00} + \frac{0.000309448}{1.5469752} + \frac{0.007}{0.716} \\
\Delta \text{Theoretical SNR} &= 9.1386 \times 10^{-5} \\
&= 9 \times 10^{-5} \text{ (1 s.f.)}
\end{aligned}$$

$$\therefore \text{Theoretical SNR} = (4.66 \pm 0.09) \times 10^{-3}$$

The theoretical and experimental values of the SNR can be plotted for comparison (see Figure 22).

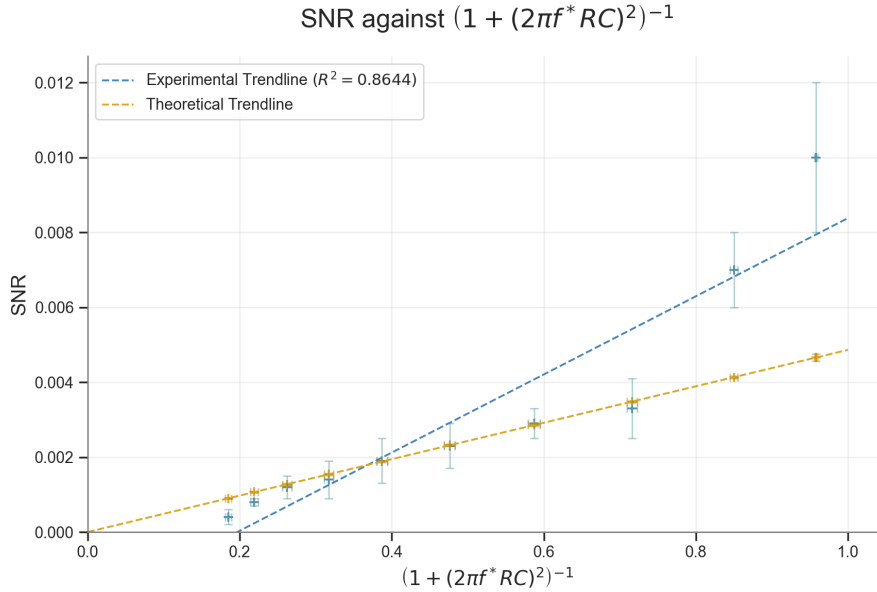


Figure 22: Comparison between theoretical and experimental SNRs, with $f^* = 100, 200, \dots, 1000$ Hz

The experimental data points corresponding to $f^* = 100, 200$ Hz are significantly higher than the theoretical prediction, and do not follow the linear trend exhibited by the other data points. Again, these two data points will be discarded for now, giving Figure 23.

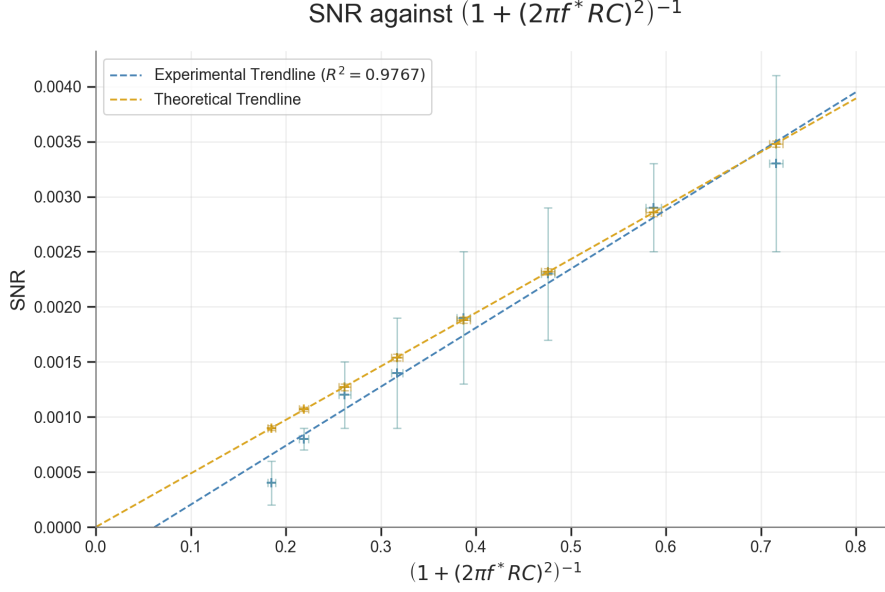


Figure 23: Comparison between theoretical and experimental SNRs, with $f^* = 300, 400, \dots, 1000$ Hz

In Figure 23, the trends are similar: both the theoretical and experimental values show a positive linear relationship between SNR and $(1 + (2\pi f^* RC)^2)^{-1}$.

However, the experimental values are often lower than the theoretical values, and a negative y-intercept is present in the experimental best-fit line, implying the presence of systematic errors leading to underestimation.

The theoretical best-fit line passes through most experimental error bars, except the two leftmost data points. This suggests that random errors may have a more significant effect on those data points, which correspond to large values of f^* .

10 Evaluation of Data

10.1 Systematic Errors

10.1.1 Underestimations

For $f^* = 300, 400, \dots, 1000$ Hz, the experimental SNRs are often less than the theoretical SNRs.

It is proposed that this is a procedural systematic error, arising due to the limitations of the apparatus. When the oscilloscope uses the FFT to compute the amplitude of the SNRs, a phenomenon known as *spectral leakage* occurs. Spectral leakage refers to the spreading of a signal's energy across multiple frequency bins in a FFT analysis, making a pure tone appear to "leak" into other frequencies (see [11]). In other words, my amplitude measurements of the sine wave signal are lower than the true value, causing an underestimation.

In practice, spectral leakage is unavoidable, as this is an inherent limitation in FFT. However, the effects of spectral leakage can be mitigated. In my experiment, I chose the default Hanning window to carry out the FFT, which balances frequency resolution and leakage reduction, and is recommended for most signals like sine

waves and narrowband random signals (see [12]). However, it is possible to choose other windows which may have a better effect, as in general different windows work best for different types of signals.

10.1.2 Overestimations

For $f^* = 100, 200$ Hz, the experimental SNRs are significantly higher than the theoretical SNRs.

It is proposed that this is a methodological systematic error, arising due to the design of data collection. Since the amplitude measurement in the FFT takes into account both the sine wave and noise components, the amplitude I recorded as the amplitude of the signal is actually the amplitude of the signal plus the amplitude of the noise at $f^* = 100, 200$ Hz. The reason that the error is so significant here, but not for higher values of f^* , is because the RC low-pass filter greatly reduces the amplitudes of high-frequency noise, so the baseline noise is negligible at high frequencies. However, $f^* = 100, 200$ Hz are low frequencies significantly below the cutoff frequency of the RC circuit, which is given (see (23) for derivation) by

$$f_C = \frac{1}{2\pi RC} = 476.5 \text{ Hz.} \quad (47)$$

This means that my measurements overestimate the amplitude of the signal at low frequencies. Thus, it seems that the average data acquisition method introduced in Section 6.2.3 is not effective, and not all of the noise is cancelled out. To rectify this error, I will instead use the "Sample" data acquisition method for the FFT. I will take a baseline noise amplitude measurement for each f^* , then subtract the baseline from the amplitude measurement used.

10.2 Random Errors

As discussed in Section 4, the instantaneous power of white noise input is random, and cannot be calculated or predicted. Therefore, all measurements related to it will vary unpredictably over time. I have tried to mitigate this effect by averaging over 5 trials, but it is impossible to eliminate this error.

11 Further Methodology

11.1 Addressing Spectral Leakage in FFT Measurement

To follow up on the issue of spectral leakage, I investigated the effect of using different windows on SNR. I also implemented the ways to reduce systematic and random errors, as entailed in Section 10.

11.2 Modified Methodology

Based on my suggestions in Section 10, I have developed the following modified methodology:

1. Set the data acquisition mode of the oscilloscope to "Sample".
2. Set the FFT window to Rectangle.
3. Set the output frequency of the sine wave signal generator to 100 Hz.
4. Wait for the FFT amplitude reading at 100 Hz to stabilize (as much as possible).
5. Record the amplitude reading.
6. Repeat Steps 3-5 for frequencies 200, 300, $\dots$, 1000 Hz.
7. Repeat Steps 2-6 four times for the Hanning, Hamming and Blackman windows.

12 Further Data Treatment

12.1 Raw Data

I used each of the Rectangle, Hanning, Hamming and Blackman windows to carry out the FFT. The data for the Rectangle window is presented in Table 8, while the rest are given in Appendix A.1. The averages are calculated the same way as Section 8.2.

f^* / Hz	Raw $V_{\text{output, signal}} / \text{V} (\pm 0.005\text{V})$					Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V} (\pm 0.005\text{V})$
	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5		
100 ± 1	0.640	0.715	0.550	0.655	0.670	0.65 ± 0.08	0.140
200 ± 1	0.510	0.560	0.590	0.545	0.535	0.55 ± 0.04	0.075
300 ± 1	0.425	0.450	0.465	0.475	0.430	0.45 ± 0.02	0.040
400 ± 1	0.410	0.360	0.405	0.425	0.410	0.40 ± 0.03	0.030
500 ± 1	0.350	0.335	0.365	0.305	0.365	0.34 ± 0.03	0.025
600 ± 1	0.320	0.345	0.340	0.305	0.330	0.33 ± 0.02	0.025
700 ± 1	0.245	0.280	0.255	0.295	0.290	0.27 ± 0.02	0.025
800 ± 1	0.245	0.250	0.235	0.220	0.260	0.24 ± 0.02	0.020
900 ± 1	0.195	0.225	0.190	0.195	0.210	0.20 ± 0.02	0.020
1000 ± 1	0.150	0.145	0.130	0.145	0.140	0.14 ± 0.01	0.020

Table 8: Raw data obtained using the Rectangle window

12.2 Processed Data

The SNRs for the Rectangle window is given in Tables 9, while the rest are given in Appendix A.2. Calculating noise-corrected $\bar{V}_{\text{output, signal}}$ is demonstrated as follows for $f^* = 100$ Hz in the Rectangle window:

$$\begin{aligned} \text{Noise-corrected } \bar{V}_{\text{output, signal}} &= \text{Raw } \bar{V}_{\text{output, signal}} - V_{\text{noise baseline}} \\ &= (0.65 \pm 0.08) \text{ V} - (0.140 \pm 0.005) \text{ V} \\ &= 0.51 \pm 0.09 \text{ V} \end{aligned}$$

Then the FFT correction factors are applied and the SNR is calculated, same as in Section 8.3.

f^* / Hz	Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V}$	Noise-corrected $\bar{V}_{\text{output, signal}}$	FFT-adjusted $\bar{V}_{\text{output, signal}}$	SNR / 10^{-3}
100 ± 1	0.65 ± 0.08	0.140 ± 0.005	0.51 ± 0.09	0.51 ± 0.09	4 ± 2
200 ± 1	0.55 ± 0.04	0.075 ± 0.005	0.48 ± 0.05	0.48 ± 0.05	4.0 ± 0.8
300 ± 1	0.45 ± 0.02	0.040 ± 0.005	0.41 ± 0.03	0.41 ± 0.03	2.9 ± 0.4
400 ± 1	0.40 ± 0.03	0.030 ± 0.005	0.37 ± 0.04	0.37 ± 0.04	2.3 ± 0.5
500 ± 1	0.34 ± 0.03	0.025 ± 0.005	0.32 ± 0.04	0.32 ± 0.04	1.8 ± 0.4
600 ± 1	0.33 ± 0.02	0.025 ± 0.005	0.30 ± 0.03	0.30 ± 0.03	1.5 ± 0.3
700 ± 1	0.27 ± 0.02	0.025 ± 0.005	0.25 ± 0.03	0.25 ± 0.03	1.1 ± 0.3
800 ± 1	0.24 ± 0.02	0.020 ± 0.005	0.22 ± 0.03	0.22 ± 0.03	0.8 ± 0.2
900 ± 1	0.20 ± 0.02	0.020 ± 0.005	0.18 ± 0.03	0.18 ± 0.03	0.6 ± 0.2
1000 ± 1	0.14 ± 0.01	0.020 ± 0.005	0.12 ± 0.02	0.12 ± 0.02	0.25 ± 0.08

Table 9: Processed data obtained using the Rectangle window

13 Further Discussion

13.1 Further Graphical Analysis and Comparison with Theoretical Model

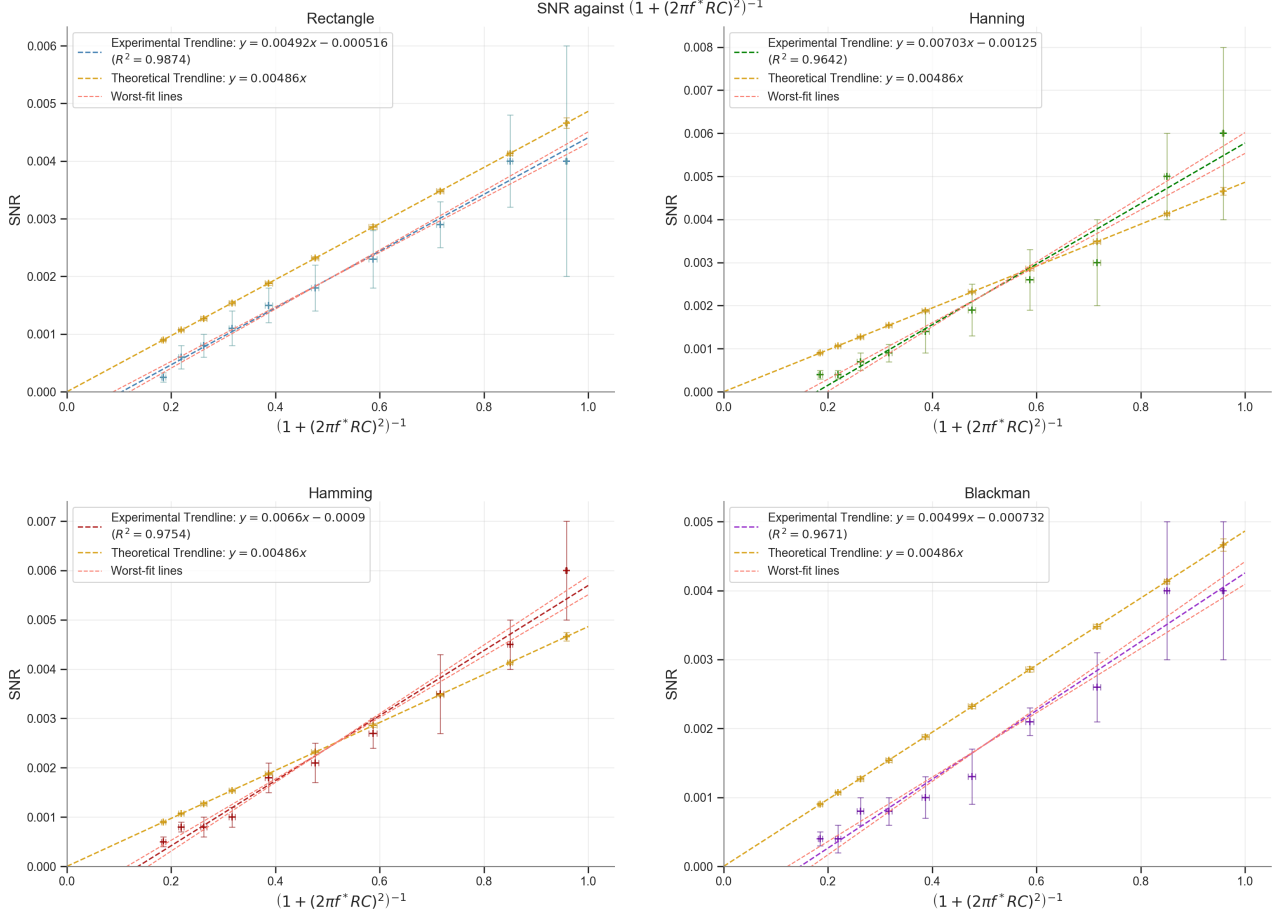


Figure 24: SNR against $(1 + (2\pi f^* RC)^2)^{-1}$ for each FFT window

The data still follows the same increasing linear trend as before, with high R^2 values. By applying the suggestions in Section 10.1.2, the data points corresponding to $f^* = 100, 200$ Hz no longer deviate significantly from the linear trendline, and the overestimation has been mitigated.

However, for Rectangle and Blackman windows, the theoretical values consistently lie above the error bars of the experimental values; implying that methodology modifications in the further investigation were unable to eliminate the existing underestimating systematic error. This is to be expected, as the FFT has inherent theoretical limits regarding accuracy, leading to systematic underestimation (see Section 10.1.1).

It is worth noting that some of the error bars shown in Figure 24 are quite large, which may be attributed to random errors.

To compare how the windows perform against each other, I plotted Figure 25.

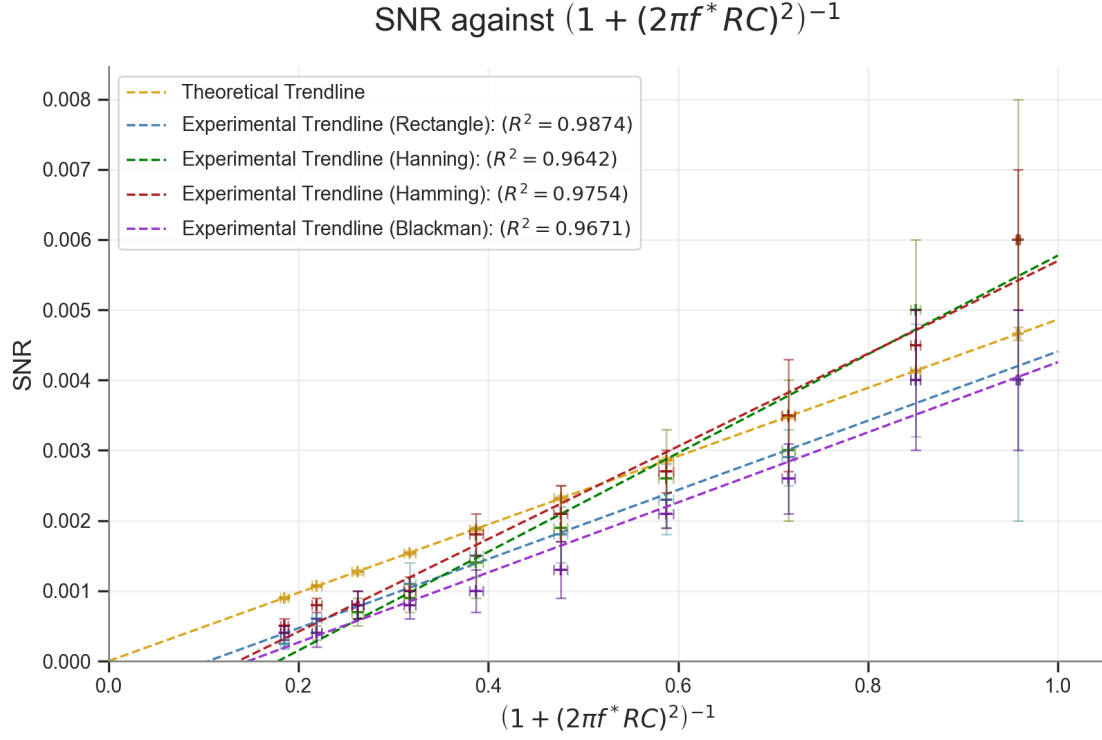


Figure 25: Comparison of SNRs from various FFT windows with theoretical model

Graphically, the theoretical slope matches the experimental slopes from the Rectangle and Blackman windows quite well, while the Hanning and Hamming experimental slopes are greater. For smaller $\left(1 + (2\pi f^* RC)^2\right)^{-1}$, the error bars from different windows are very close together and quite small, while the error bars at higher $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ become quite large (and the data points become vertically spread apart), indicating that the effects of random errors become more significant at lower frequencies of f^* .

13.2 Further Evaluation

Table 10 shows the theoretical and experimental slopes and y-intercepts given by Excel LINEST, visualized in Figures 26, 27.

Window Type	Slope / 10^{-3}	y-intercept / 10^{-3}
Theoretical	4.862 ± 0.005	0.001 ± 0.003
Rectangle	4.9 ± 0.2	-0.5 ± 0.1
Hanning	7.0 ± 0.5	-1.3 ± 0.3
Hamming	6.6 ± 0.4	-0.9 ± 0.2
Blackman	5.0 ± 0.3	-0.7 ± 0.1

Table 10: Comparison of slopes and y-intercepts

There are uncertainties in the theoretical slope and y-intercept, because there are uncertainties in R , C , etc.

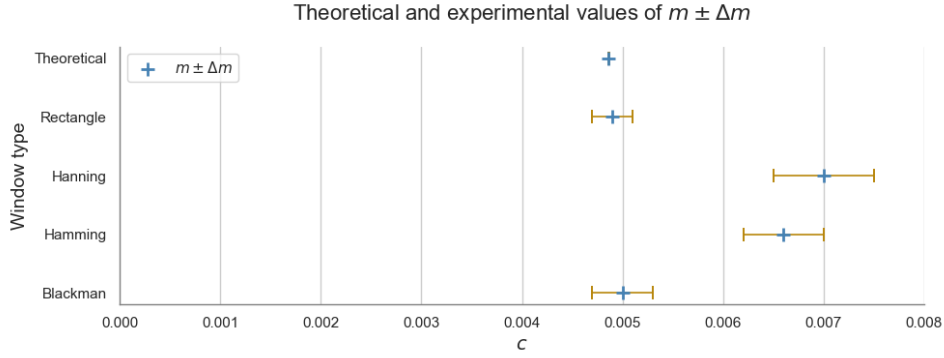


Figure 26: Theoretical and experimental values of $m \pm \Delta m$

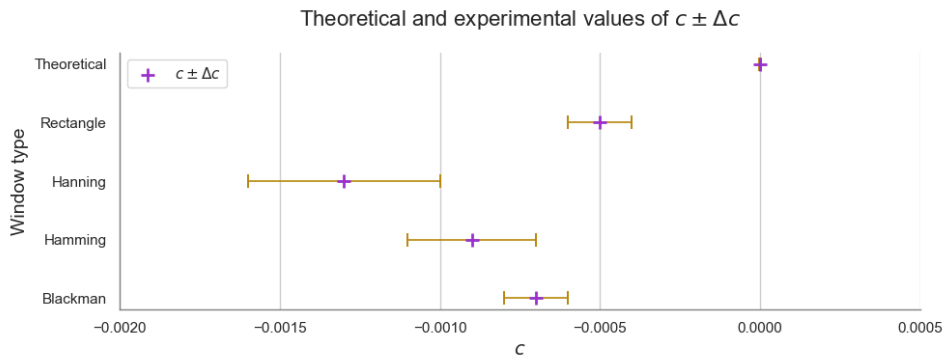


Figure 27: Theoretical and experimental values of $c \pm \Delta c$

Figure 26 shows that the theoretical slope falls in the uncertainty range of the experimental slopes under the Rectangle and Blackman windows, but not the Hanning and Hamming windows; while Figure 27 shows that the theoretical y-intercept does not fall in the uncertainty range of any experimental y-intercept.

In particular, the Hanning window seems to perform particularly poorly, with its experimental slope and y-intercept both furthest from theoretical predictions.

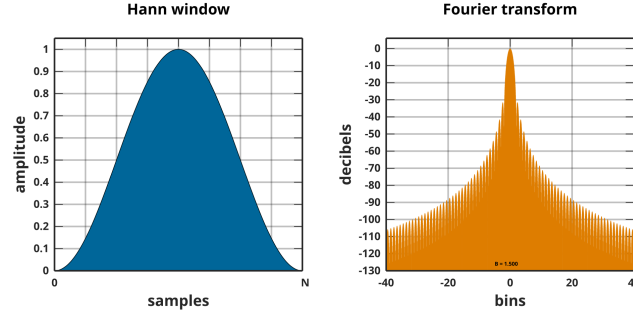


Figure 28: FFT of a pure sine wave, using the Hanning window (taken from [7])

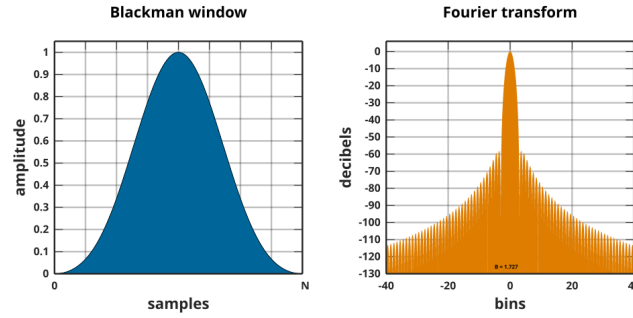


Figure 29: FFT of a pure sine wave, using the Blackman window (taken from [7])

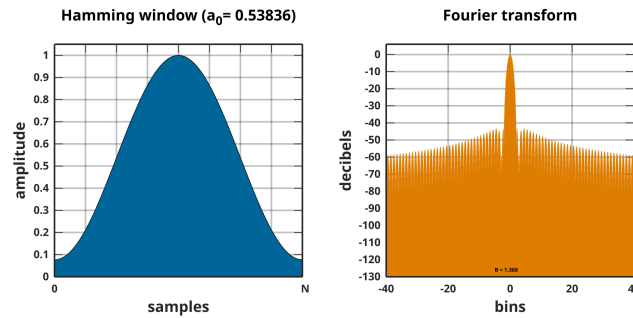


Figure 30: FFT of a pure sine wave, using the Hamming window (taken from [7])

A partial explanation of why the Hanning window is inferior to the Blackman window comes from the spectral leakage plots of the windows, i.e. the graphs on the right in Figures 28 and 29. The graphs show the maximum scalloping loss (a type of spectral leakage) for the two windows, which occurs when the signal's frequency is exactly halfway between two FFT bins.

Graphically, the Blackman window has smaller peaks right next to it, compared to the Hanning window. In particular, the third and fourth sidelobes of the Blackman window are zeroed (see [13]), reducing spectral leakage and making it better than the Hanning window. The sidelobes further out are much less significant, as the y-axis is in decibels, a log scale, so the remaining lobes have little effect.

In fact, the Hamming and Hanning windows are both part of a larger family of FFT windows, called raised-cosine windows (see [13]), and share similar properties, explaining why the Hamming window (see Figure 30) is also worse than the Blackman window.

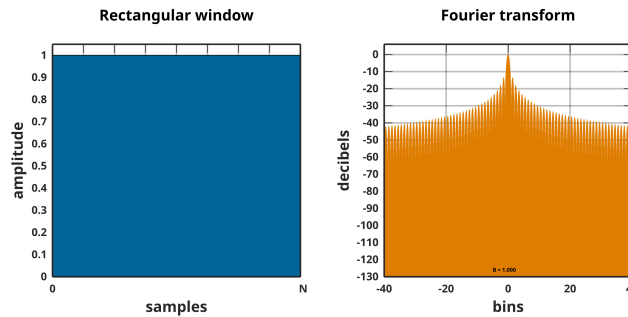


Figure 31: FFT of a pure sine wave, using the Rectangle window (taken from [7])

Nevertheless, this does not explain why the Rectangle window (see Figure 31) outperforms the Hanning and Hamming windows. The Rectangle window has the greatest sidelobes, leading to maximum scalloping loss, and is worse than the Hanning and Hamming windows in this aspect. This signals that properties of the window, other than spectral leakage, may influence the results, with potential for future study.

14 Numerical Simulation

The experimental results support the theoretical model well. To further confirm the theory, I will use numerical simulation to verify (39).

In Section 5, the theoretical model was derived using the time-domain differential equation

$$v_{\text{in}}(t) = RC \frac{dv_{\text{out}}(t)}{dt} + v_{\text{out}}(t) \quad (48)$$

i.e. (16). Then I proceeded to convert it to the frequency domain and solve it. To verify this, I will employ numerical simulation methods to solve the time-domain differential equation and analyse the SNRs.

14.1 Methodology

I will use Python for the numerical simulation, as Python is good at handling data. The white noise generation and FFT are available through standard Python tools. The white noise is added to a sinusoidal signal of frequency f^* to give $v_{\text{in}}(t)$, i.e.

$$v_{\text{noise}}(t) + V_0 \sin(2\pi f^* t) = RC \frac{dv_{\text{out}}(t)}{dt} + v_{\text{out}}(t) \quad (49)$$

There are three main steps:

1. Generate Gaussian white noise.
2. Numerically solve the differential equation for $v_{\text{out}}(t)$.
3. Perform a FFT on $v_{\text{out}}(t)$ to find the SNRs.

I chose the 4th-order Runge-Kutta method for the numerical solver.

With numerical simulation, I need to choose a sampling rate f_s , i.e. number of samples per second (see [14]). According to the Nyquist-Shannon sampling theorem (see [15]), this introduces a bandwidth $B = \frac{f_s}{2}$.

For convenience in computing the FFT, f_s is often chosen to be a power of two. I chose $f_s = 8192$ Hz, giving $B = 4096$ Hz. When I simulated the behaviour of the RC circuit, I allowed some time at the beginning for it to settle into the steady-state, which was removed, and performed the FFT on the remaining period.

I arbitrarily set $R = 1000 \Omega$, $C = 1 \mu\text{F}$, $V_0 = 6 \text{ V}$, $N_0 = 0.0004 \text{ V}^2 \text{ Hz}^{-1}$. For the values of f^* , I chose them such that they would be evenly spaced on the x-axis, $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ between 0 and 1.

14.2 Numerical Simulation, Graphical Analysis, and Evaluation

The Python code is given in Appendix B. The processed data is given in Table 11, and shown in Figure 32.

$\left(1 + (2\pi f^* RC)^2\right)^{-1}$	Numerical SNR (10 s.f.)	Theoretical SNR
0.05	19.54017373	18
0.10	37.36558846	36
0.15	56.05431374	54
0.20	73.53308595	72
0.25	91.11864264	90
0.30	108.6478109	108
0.35	127.6315794	126
0.40	143.8175909	144
0.45	159.5928345	162
0.50	175.8474469	180

Table 11: Numerical and theoretical SNRs

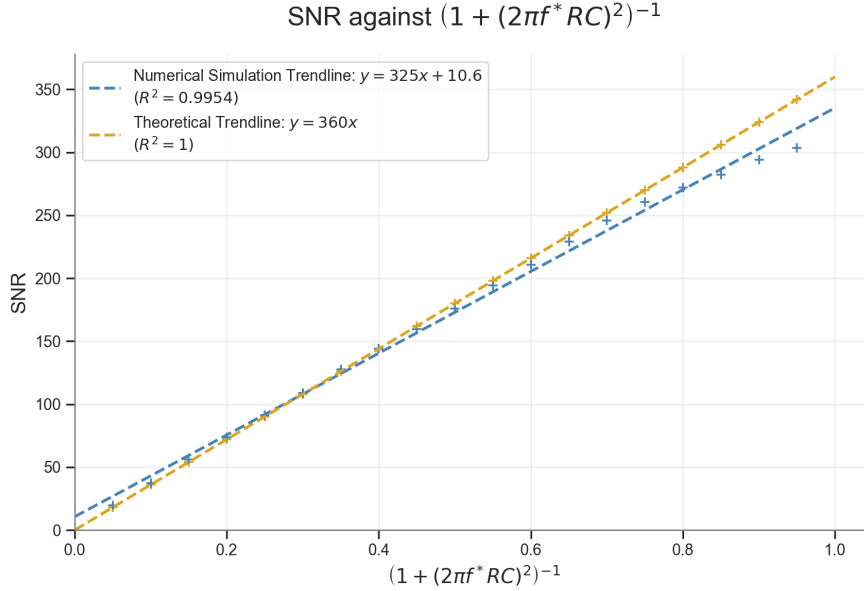


Figure 32: SNR against $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ from numerical simulation

There is a high R^2 value of 0.9954, with a positive linear trend. For small $\left(1 + (2\pi f^* RC)^2\right)^{-1}$ (i.e. high f^*), the numerical and theoretical SNRs closely agree. However, the numerical SNRs are lower than theoretical SNRs for large $\left(1 + (2\pi f^* RC)^2\right)^{-1}$.

Since there are no experimental errors here, FFT-related errors are the only source of errors. It is proposed that this underestimating systematic error is spectral leakage (explained in Section 10.1.1). This affirms that FFT-related errors must be contributing to the errors found in the further investigation in Section 13, though this does not rule out the presence of other experimental underestimating systematic errors.

In general, the theoretical and numerical SNRs are quite similar, with similar slopes, and the numerical y-intercept is very small, supporting the theory.

15 Conclusion

I addressed the research question: *How does the frequency of an input sine wave f^* , with background white noise, affect the SNR after noise reduction through a RC low-pass filter?*

In the theoretical model in Section 5, I found SNR in terms of f^* as

$$\text{SNR} = \frac{2\pi RC V_0^2}{N_0 \arctan(2\pi BRC) \left(1 + (2\pi f^* RC)^2\right)}. \quad (50)$$

I experimentally verified this relationship in Sections 6 to 10. Moreover, to address the errors discussed in Section 10, I refined my methodology in Section 10, which reduced overestimating systematic errors, and investigated the effect of FFT windows on the experimental SNR in Sections 11 to 13, finding that the Blackman and Rectangle windows perform the best for this experiment. I also simulated the experiment numerically in Section 14, and confirmed spectral leakage errors as a key contributing factor to underestimating systematic errors in the experimental sections.

To further improve my experiment, I may address random errors by carrying out more trials per data point, so I may use standard deviation (as opposed to half-range) for uncertainty analysis.

This investigation focused on low-pass RC filters only. A natural extension is to investigate other filters, e.g. Sallen-Key filters, which exhibit resonance behaviour (see [16]). Another possible extension is to consider other types of noise, e.g. pink noise, where PSD is inversely proportional to frequency (see [17]).

References

- [1] A. Devices, “Low-pass filter | analog devices,” [Analog.com](#), 2024.
- [2] [Oklahoma State University](#), Jan 2003.
- [3] [resources.pcb.cadence.com](#), 2020.
- [4] M. L. Boas, [Mathematical Methods in the Physical Sciences.](#), vol. 74. Oct 1967.
- [5] [Keysight.com](#), 2024.
- [6] Ar501, “Fast fourier transforms explained,”
- [7] Dec 2020.
- [8] [Siemens.com](#), 2025.
- [9] M. Klein, “Designing a white, pink & blue noise generator from scratch,” Jul 2022.
- [10] S. Pigeon, “The ultimate white noise generator • design your own color,” 2013.
- [11] B. McFee, “6.4. spectral leakage and windowing — digital signals theory,” [brianmcfee.net](#), 2020.
- [12] N. Instruments, “Understanding ffts and windowing overview,”
- [13] F. Harris, “On the use of windows for harmonic analysis with the discrete fourier transform,” [Proceedings of the IEEE](#), vol. 66, no. 1, p. 51–83, 1978.
- [14] [Analog.com](#), 2024.
- [15] R. Keim, “The nyquist–shannon theorem: Understanding sampled systems - technical articles,” [www.allaboutcircuits.com](#), May 2020.
- [16] E. Tutorials, “Sallen and key filter design for second order rc filters,” [Basic Electronics Tutorials](#), Jul 2019.
- [17] A. Sarow, “White, brown, pink and green noise: Benefits of each,” [www.soundly.com](#), Jun 2024.

A Data Tables

A.1 Further Raw Data

Here are the raw data tables for Section 12.1, showing raw data obtained using the Hanning, Hamming and Blackman windows.

f^* / Hz	Raw $V_{\text{output, signal}} / \text{V} (\pm 0.005\text{V})$					Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V} (\pm 0.005\text{V})$
	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5		
100 ± 1	0.450	0.405	0.375	0.440	0.385	0.41 ± 0.04	0.060
200 ± 1	0.355	0.355	0.315	0.380	0.365	0.35 ± 0.03	0.030
300 ± 1	0.255	0.290	0.300	0.260	0.245	0.27 ± 0.03	0.010
400 ± 1	0.265	0.255	0.225	0.230	0.260	0.25 ± 0.02	0.010
500 ± 1	0.195	0.220	0.180	0.195	0.220	0.20 ± 0.02	0.005
600 ± 1	0.190	0.180	0.200	0.185	0.165	0.18 ± 0.02	0.005
700 ± 1	0.155	0.140	0.135	0.150	0.160	0.15 ± 0.01	0.005
800 ± 1	0.140	0.115	0.135	0.120	0.125	0.13 ± 0.01	0.005
900 ± 1	0.110	0.110	0.110	0.095	0.110	0.107 ± 0.007	0.005
1000 ± 1	0.100	0.090	0.095	0.100	0.105	0.098 ± 0.007	0.005

Table 12: Raw data obtained using the Hanning window

f^* / Hz	Raw $V_{\text{output, signal}} / \text{V} (\pm 0.005\text{V})$					Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V} (\pm 0.005\text{V})$
	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5		
100 ± 1	0.410	0.405	0.385	0.425	0.400	0.40 ± 0.02	0.045
200 ± 1	0.340	0.330	0.340	0.355	0.345	0.34 ± 0.01	0.020
300 ± 1	0.310	0.315	0.280	0.300	0.310	0.30 ± 0.02	0.020
400 ± 1	0.270	0.270	0.260	0.265	0.255	0.264 ± 0.008	0.010
500 ± 1	0.225	0.225	0.220	0.230	0.240	0.23 ± 0.01	0.010
600 ± 1	0.220	0.200	0.220	0.215	0.215	0.21 ± 0.01	0.010
700 ± 1	0.160	0.155	0.160	0.160	0.145	0.156 ± 0.008	0.005
800 ± 1	0.140	0.140	0.145	0.145	0.150	0.144 ± 0.005	0.005
900 ± 1	0.130	0.140	0.130	0.125	0.135	0.132 ± 0.008	0.005
1000 ± 1	0.120	0.115	0.120	0.120	0.120	0.119 ± 0.002	0.005

Table 13: Raw data obtained using the Hamming window

f^* / Hz	Raw $V_{\text{output, signal}} / \text{V} (\pm 0.005\text{V})$					Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V} (\pm 0.005\text{V})$
	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5		
100 ± 1	0.325	0.310	0.270	0.270	0.295	0.29 ± 0.03	0.035
200 ± 1	0.270	0.255	0.270	0.280	0.250	0.27 ± 0.02	0.020
300 ± 1	0.230	0.220	0.230	0.225	0.205	0.22 ± 0.01	0.020
400 ± 1	0.180	0.185	0.190	0.185	0.195	0.187 ± 0.008	0.010
500 ± 1	0.160	0.160	0.140	0.140	0.145	0.15 ± 0.01	0.010
600 ± 1	0.140	0.135	0.135	0.115	0.120	0.13 ± 0.01	0.010
700 ± 1	0.125	0.110	0.120	0.110	0.125	0.118 ± 0.007	0.005
800 ± 1	0.115	0.115	0.110	0.115	0.110	0.113 ± 0.003	0.005
900 ± 1	0.090	0.095	0.105	0.095	0.080	0.09 ± 0.01	0.005
1000 ± 1	0.085	0.090	0.085	0.085	0.085	0.086 ± 0.002	0.005

Table 14: Raw data obtained using the Blackman window

A.2 Further Processed Data

Here are the processed data tables for Section 12.2, showing processed data obtained using the Hanning, Hamming and Blackman windows.

f^* / Hz	Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V}$	Noise-corrected $\bar{V}_{\text{output, signal}}$	FFT-adjusted $\bar{V}_{\text{output, signal}}$	SNR / 10^{-3}
100 ± 1	0.41 ± 0.04	0.060 ± 0.005	0.35 ± 0.05	0.57 ± 0.08	6 ± 2
200 ± 1	0.35 ± 0.03	0.030 ± 0.005	0.32 ± 0.04	0.52 ± 0.07	5 ± 1
300 ± 1	0.27 ± 0.03	0.010 ± 0.005	0.26 ± 0.04	0.42 ± 0.07	3 ± 1
400 ± 1	0.25 ± 0.02	0.010 ± 0.005	0.24 ± 0.03	0.39 ± 0.05	2.6 ± 0.7
500 ± 1	0.20 ± 0.02	0.005 ± 0.005	0.20 ± 0.03	0.33 ± 0.05	1.9 ± 0.6
600 ± 1	0.18 ± 0.02	0.005 ± 0.005	0.18 ± 0.03	0.29 ± 0.05	1.4 ± 0.5
700 ± 1	0.15 ± 0.01	0.005 ± 0.005	0.14 ± 0.02	0.23 ± 0.03	0.9 ± 0.2
800 ± 1	0.13 ± 0.01	0.005 ± 0.005	0.12 ± 0.02	0.20 ± 0.03	0.7 ± 0.2
900 ± 1	0.107 ± 0.007	0.005 ± 0.005	0.10 ± 0.01	0.16 ± 0.02	0.4 ± 0.1
1000 ± 1	0.098 ± 0.007	0.005 ± 0.005	0.09 ± 0.01	0.15 ± 0.02	0.4 ± 0.1

Table 15: Processed data obtained using the Hanning window

f^* / Hz	Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V}$	Noise-corrected $\bar{V}_{\text{output, signal}}$	FFT-adjusted $\bar{V}_{\text{output, signal}}$	SNR / 10^{-3}
100 ± 1	0.40 ± 0.02	0.045 ± 0.005	0.36 ± 0.03	0.57 ± 0.05	6 ± 1
200 ± 1	0.34 ± 0.01	0.020 ± 0.005	0.32 ± 0.02	0.51 ± 0.03	4.5 ± 0.5
300 ± 1	0.30 ± 0.02	0.020 ± 0.005	0.28 ± 0.03	0.45 ± 0.05	3.5 ± 0.8
400 ± 1	0.264 ± 0.008	0.010 ± 0.005	0.25 ± 0.01	0.40 ± 0.02	2.7 ± 0.3
500 ± 1	0.23 ± 0.01	0.010 ± 0.005	0.22 ± 0.02	0.35 ± 0.03	2.1 ± 0.4
600 ± 1	0.21 ± 0.01	0.010 ± 0.005	0.20 ± 0.02	0.32 ± 0.03	1.8 ± 0.3
700 ± 1	0.156 ± 0.008	0.005 ± 0.005	0.15 ± 0.01	0.24 ± 0.02	1.0 ± 0.2
800 ± 1	0.144 ± 0.005	0.005 ± 0.005	0.14 ± 0.01	0.22 ± 0.02	0.8 ± 0.2
900 ± 1	0.132 ± 0.008	0.005 ± 0.005	0.13 ± 0.01	0.21 ± 0.02	0.8 ± 0.1
1000 ± 1	0.119 ± 0.002	0.005 ± 0.005	0.11 ± 0.01	0.17 ± 0.02	0.5 ± 0.1

Table 16: Processed data obtained using the Hamming window

f^* / Hz	Raw $\bar{V}_{\text{output, signal}} / \text{V}$	$V_{\text{noise baseline}} / \text{V}$	Noise-corrected $\bar{V}_{\text{output, signal}}$	FFT-adjusted $\bar{V}_{\text{output, signal}}$	SNR / 10^{-3}
100 ± 1	0.29 ± 0.03	0.035 ± 0.005	0.26 ± 0.04	0.51 ± 0.08	4 ± 1
200 ± 1	0.27 ± 0.02	0.020 ± 0.005	0.25 ± 0.03	0.49 ± 0.06	4 ± 1
300 ± 1	0.22 ± 0.01	0.020 ± 0.005	0.20 ± 0.02	0.39 ± 0.04	2.6 ± 0.5
400 ± 1	0.187 ± 0.008	0.010 ± 0.005	0.18 ± 0.01	0.35 ± 0.02	2.1 ± 0.2
500 ± 1	0.15 ± 0.01	0.010 ± 0.005	0.14 ± 0.02	0.28 ± 0.04	1.3 ± 0.4
600 ± 1	0.13 ± 0.01	0.010 ± 0.005	0.12 ± 0.02	0.24 ± 0.04	1.0 ± 0.3
700 ± 1	0.118 ± 0.007	0.005 ± 0.005	0.11 ± 0.01	0.22 ± 0.02	0.8 ± 0.2
800 ± 1	0.113 ± 0.003	0.005 ± 0.005	0.11 ± 0.01	0.22 ± 0.02	0.8 ± 0.2
900 ± 1	0.09 ± 0.01	0.005 ± 0.005	0.08 ± 0.02	0.16 ± 0.04	0.4 ± 0.2
1000 ± 1	0.086 ± 0.002	0.005 ± 0.005	0.08 ± 0.01	0.16 ± 0.02	0.4 ± 0.1

Table 17: Processed data obtained using the Blackman window

B Python Code for Numerical Simulation

B.1 White Noise Generation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

def generate_gaussian_white_noise(duration, sample_rate, mean=0, std=1, seed=None):
    if seed is not None:
        np.random.seed(seed)

    num_samples = int(duration * sample_rate)
    # time = np.linspace(0, duration, num_samples, endpoint=False)
    noise = np.random.normal(mean, std, num_samples)

    return noise
```

B.2 Differential Equation Numerical Solver and FFT

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

from whitenoise import generate_gaussian_white_noise

def solve_rk4(v_in, time, R, C):
    """
    Super simple RK4 for RC circuit.
    v_in = R*C * dv_out/dt + v_out
    """
    tau = R * C
    dt = time[1] - time[0]
    v_out = np.zeros_like(v_in)

    for i in range(len(time) - 1):
        # Current v_out and v_in
        v = v_out[i]
        v_in_current = v_in[i]

        # RK4 coefficients
        k1 = (v_in_current - v) / tau
        k2 = (v_in_current - (v + dt*k1/2)) / tau
        k3 = (v_in_current - (v + dt*k2/2)) / tau
        k4 = (v_in_current - (v + dt*k3)) / tau

        # Update
        v_out[i+1] = v + dt * (k1 + 2*k2 + 2*k3 + k4) / 6

    return v_out

def H(f): # transfer function
    return 1/np.sqrt(1+2*np.pi*f*C*R)
```

```

def find_snrs(duration, sample_rate, fstar, R, C, V_0, N_0, B):
    time = np.linspace(0, duration, duration*sample_rate)

    sine_in = V_0 * np.sin(2*np.pi * time * fstar)

    v_in = sine_in + noise_in
    v_out = solve_rk4(v_in, time, R=R, C=C)

    v_out = v_out[len(v_out)//4:]

    desired_resolution = fstar / 10

    nperseg_min = int(sample_rate / desired_resolution)
    nperseg = min(nperseg_min, len(v_out) // 2)

    freqs, psd_out = signal.welch(v_out, sample_rate, nperseg=nperseg)
    delta_f = freqs[1] - freqs[0]

    total_power_psd = np.sum(psd_out) * delta_f

    signal_bw = delta_f*2
    mask = ~((freqs > fstar - signal_bw) & (freqs < fstar + signal_bw))
    noise_power_psd = np.sum(psd_out[mask]) * delta_f
    signal_power_psd = total_power_psd - noise_power_psd

    snr_numerical = signal_power_psd / noise_power_psd

    print(f"Numerical output signal power: {signal_power_psd}\nNumerical
          output noise power: {noise_power_psd}\nNumerical SNR: {snr_numerical}")

    theoretical_outsig = V_0**2/2 / (1+(2*np.pi * fstar * R * C)**2)
    theoretical_outnoi = N_0 * np.arctan(2*np.pi*B*R*C) / (4 * R * C)

    theoretical_SNR = 4 * R * C * V_0**2 / (N_0 * (1+(2*np.pi * fstar * R * C)**2))

    print(f"Theoretical output signal power: {theoretical_outsig}\nTheoretical
          output noise power: {theoretical_outnoi}\nTheoretical SNR:
          {theoretical_SNR}")

    return float(snr_numerical), float(theoretical_SNR)

duration = 10 # s
sample_rate = 8192 # Hz

```

```

R = 1000 # ohm
C = 1e-6 # F
V_0 = 6 # V
N_0 = 0.00040 # V2 Hz-1

B = sample_rate / 2

x = np.linspace(0.05, 0.95, 19)
x = np.sqrt(1 / x - 1) / (2*np.pi*R*C)

fstars = []
nSNRs = []
tSNRs = []

noise_in = generate_gaussian_white_noise(duration, sample_rate, seed=1)

for fstar in x:
    numerical_SNR, theoretical_SNR = find_snrs(duration=duration,
        sample_rate=sample_rate, fstar=fstar, R=R, C=C, V_0=V_0, N_0=N_0, B=B)

    fstars.append(fstar)
    nSNRs.append(numerical_SNR)
    tSNRs.append(theoretical_SNR)

fstars = np.array(fstars)
nSNRs = np.array(nSNRs)
tSNRs = np.array(tSNRs)

np.save("fstars.npy", fstars)
np.save("nSNRs.npy", nSNRs)
np.save("tSNRs.npy", tSNRs)

```